

Ciencia e Ingeniería en el estudio de la mente –

Lección 2. Resolución de Problemas y Búsqueda

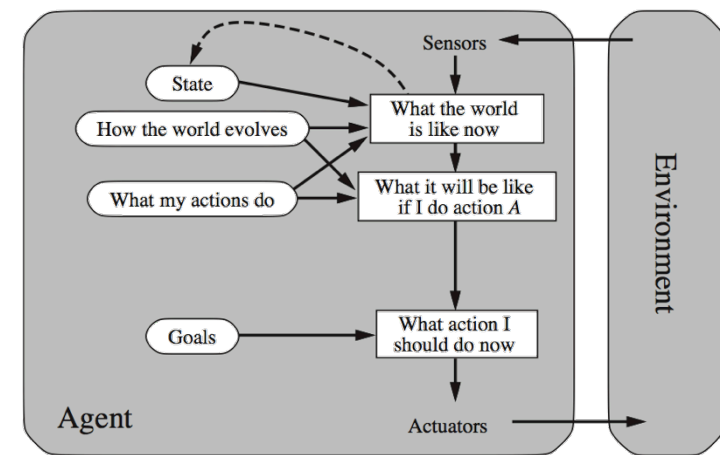
Índice

- Agente de **Resolución de Problemas**
 - Agente basado en objetivos
- **Formulación** del problema
 - Problemas ejemplo
- Algoritmos de Búsqueda básicos
 - Ciegos / **Sin información**

Resolución de problemas

- La resolución de problemas es una capacidad que consideramos inteligente
- P. e. Encontrar el camino en un laberinto.
Resolver un crucigrama. Jugar a un juego.
- ... El objetivo es que un programa también sea capaz de resolverlos

Agente de resolución de problemas



- Los agentes más sencillos son agentes reflejos que asocian una acción a cada estado.
 - Estos agentes no son viables cuando hay demasiados estados y/o el proceso de aprendizaje llevaría mucho tiempo.
- Los agentes de **resolución de problemas** son **agentes basados en objetivos**
 - Las acciones van dirigidas a conseguir un objetivo en el futuro.

Formalización Resolución

Problemas (**espacio de estados**)

■ Modelo de Estados

- Espacio de estados finito y discreto S
- Estado inicial $s_0 \in S$
- Un conjunto de estados objetivos $G \subseteq S$
- Acciones aplicables $A(s) \subseteq A$ en cada estado $s \in S$
- Una función transición $f(s, a)$ para $s \in S$ y $a \in A(s)$
- Y una función de coste $c(a, s) > 0$

- **Una solución** es una secuencia aplicable de acciones $a_i, i = 0, \dots, n$, que lleva desde el estado inicial s_0 al estado objetivo $s \in SG$; es decir, $s_{n+1} \in SG$ y para $i = 0, \dots, n$,

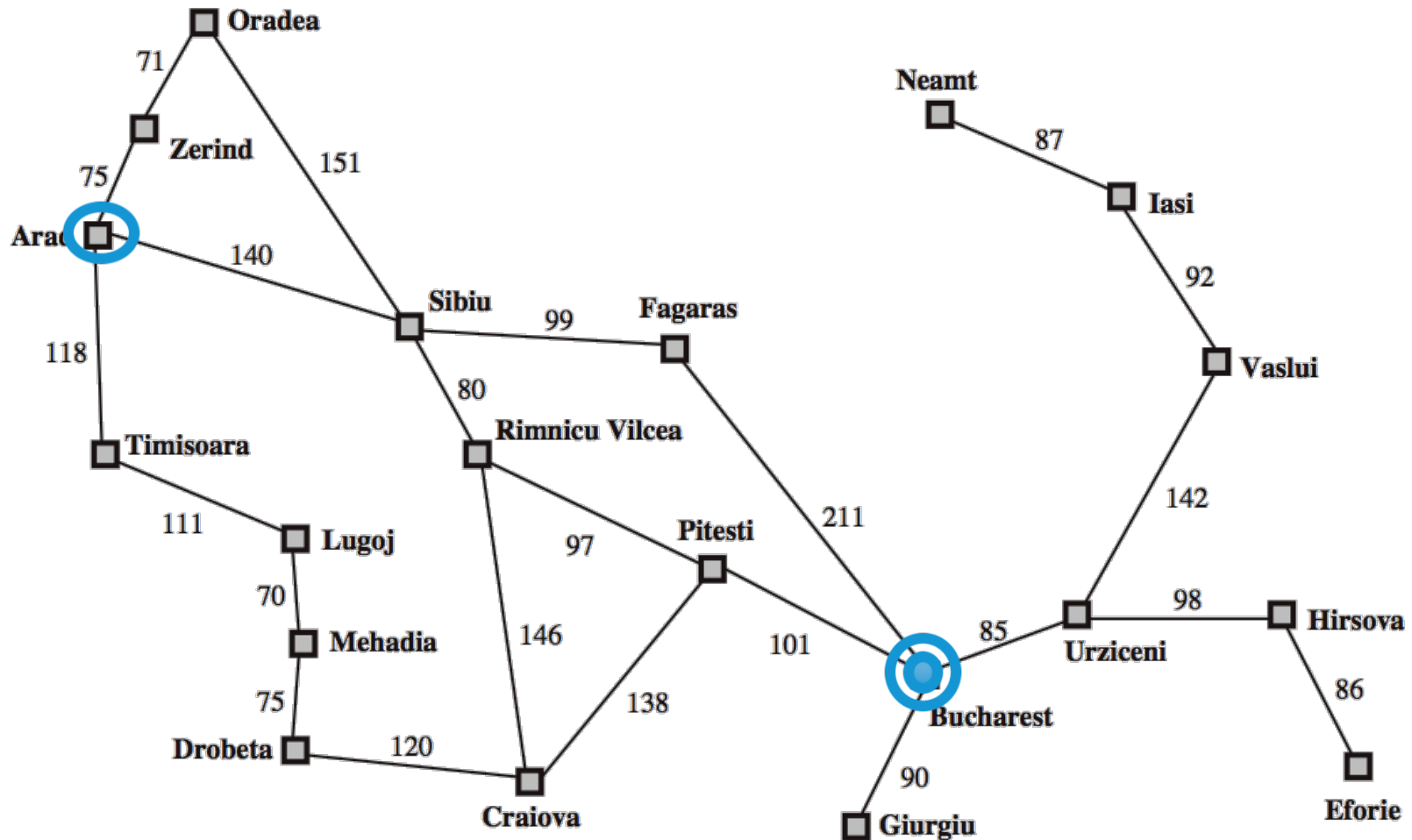
$$s_{i+1} = f(a_i, s_i) \text{ y } a_i \in A(s_i)$$

- **La solución óptima** minimiza el coste $\sum_{i=0}^n c(a_i, s_i)$

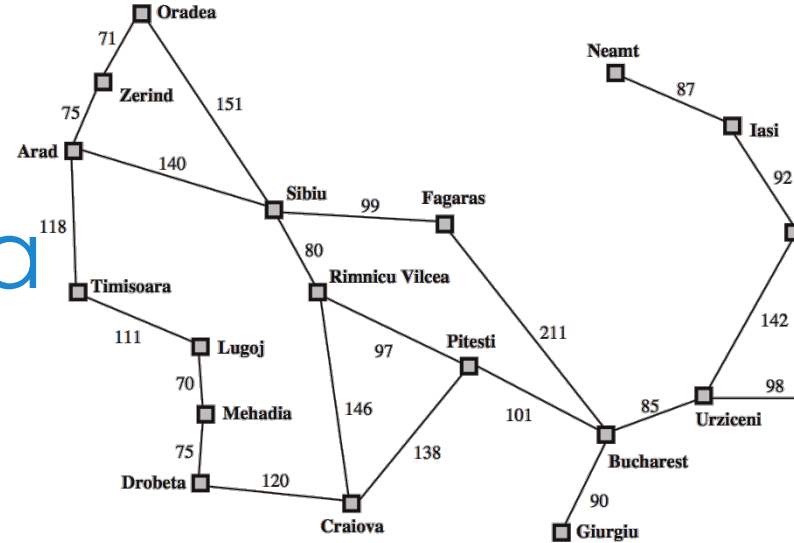
Ejemplo : Rumanía

- De vacaciones en Rumanía; estamos en Arad
 - Nuestro vuelo sale mañana de Bucharest
- Definimos Objetivo
 - Estar en Bucharest
- Definimos problema
 - Estados: varias ciudades
 - Acciones: conducir entre ciudades
- Solución
 - Secuencia de ciudades; p.e. Arad, Sibiu, Fagaras, Bucharest, ...

Ejemplo: Rumanía



Formulación Problema Rumanía



- **Estado inicial** $En(Arad)$
- **Acciones aplicables**, p.e. $En(Arad)$ las acciones aplicables son $\{lr(Sibiu), lr(Timisoara), lr(Zerin)\}$
- **El modelo de transición**, p.e.
 $Result(In(Arad), lr(Zerind)) = In(Zerind)$
- **Test Objetivo**, puede ser
 - Explícito, p.e $\{En(Bucharest)\}$
 - Implícito, p.e. $JaqueMate(x)$, $NoSucio(x)$
- **Coste del camino** (aditivo), p.e. **suma de distancias**, número de acciones ejecutadas, ...
 - $c(x,a,y)$ es el coste por paso, asumimos que es ≥ 0

Ejemplo: 8-puzzle

- **Estados:**

Localización de cada casilla

- **Estado Inicial:**

Cualquier estado puede ser inicial

7	2	4
5		6
8	3	1

Estado inicial

	1	2
3	4	5
6	7	8

Estado objetivo

- **Acciones:** Mover el hueco **Arriba, Abajo, Izquierda, Derecha**
- **Modelo Transiciones:** Dado el estado y la acción resulta el estado resultante. P.e. Si aplicamos izquierda al estado inicial tenemos el estado resultante de intercambiar el 5 y el hueco.
- **Test Objetivo:** Comprueba si es el estado final
- **Coste camino:**

Ejemplo: 8-puzzle

- **Estados:**

Localización de cada casilla

- **Estado Inicial:**

Cualquier estado puede ser inicial

7	2	4
5		6
8	3	1

Estado inicial

	1	2
3	4	5
6	7	8

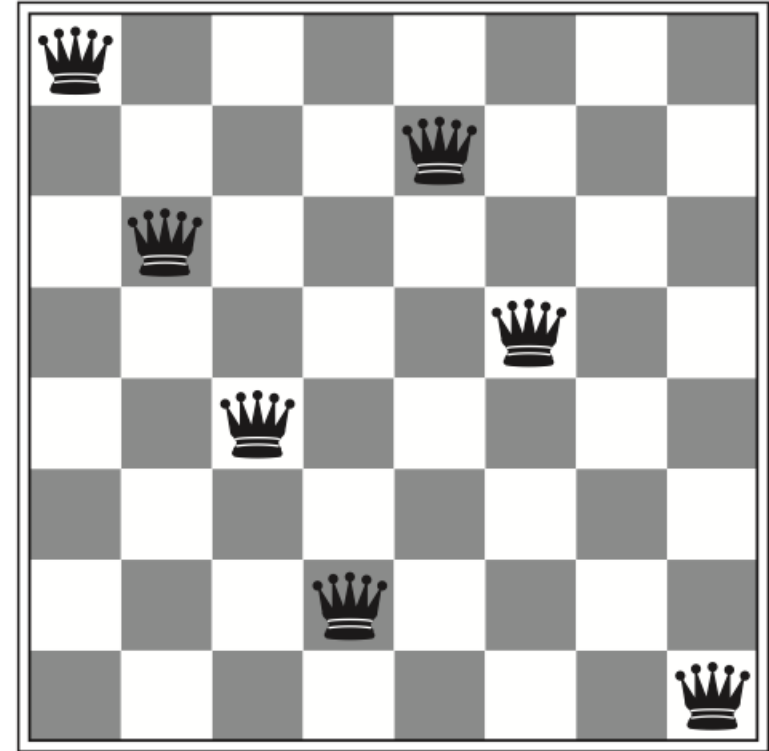
Estado objetivo

- **Acciones:** Mover el hueco **Arriba, Abajo, Izquierda, Derecha**
- **Modelo Transiciones:** Dado el estado y la acción resulta el estado resultante. P.e. Si aplicamos izquierda al estado inicial tenemos el estado resultante de intercambiar el 5 y el hueco.
- **Test Objetivo:** Comprueba si es el estado final
- **Coste camino:** Cada paso coste 1. Coste = longitud del camino.

Ejemplo: 8-reinas

Primera versión incremental. Parto del tablero vacío y coloco una reina cada vez.

- **Estados:** Cualquier combinación de 8 reinas en el tablero
- **Estado Inicial:** Tablero vacío
- **Acciones:** Añadir una reina en un cuadrado vacío
- **Modelo Transiciones:** Devuelve el tablero con la reina añadida en el cuadro especificado.
- **Test Objetivo:** 8 reinas en el tablero sin atacarse
- **Coste camino:** No hay coste. Sólo nos interesa el estado final.

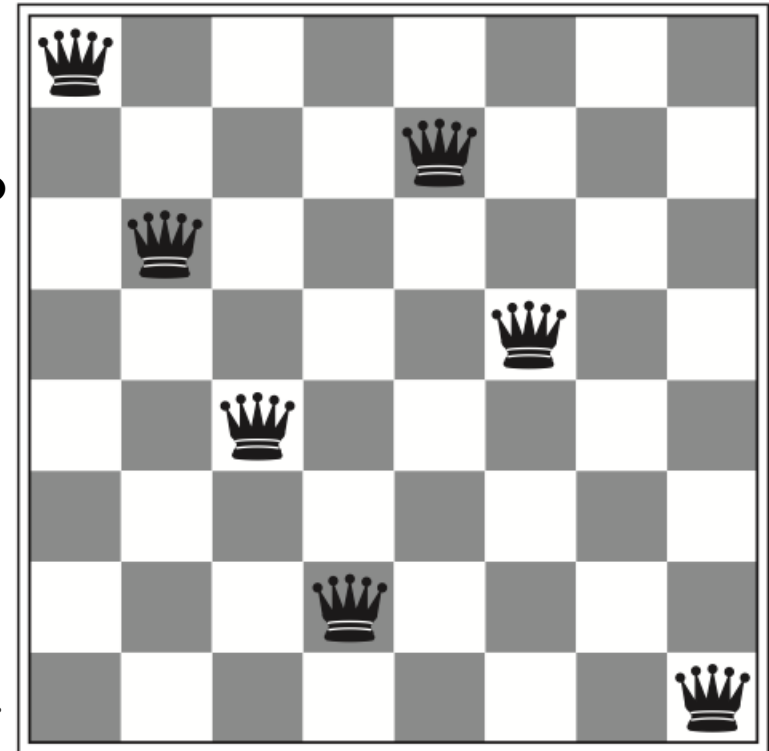


Espacio Estados
 $64 \times 63 \times \dots \times 57 = 3 \times 10^{14}$ estados

Ejemplo: 8-reinas

Segunda versión incremental. Parto del tablero vacío y coloco una reina cada vez en una columna de izquierda a derecha

- **Estados:** Cualquier combinación de 8 reinas en el tablero
- **Estado Inicial:** Tablero vacío
- **Acciones:** Añadir una reina en cualquier cuadrado de la columna libre más a la izquierda en la que no es atacada.
- **Modelo Transiciones:** Devuelve el tablero con la reina añadida en el cuadro especificado
- **Test Objetivo:** 8 reinas en el tablero sin atacarse
- **Coste camino:** No hay coste. Sólo nos interesa el estado final.



Espacio Estados
2057 estados

Algoritmos básicos de búsqueda

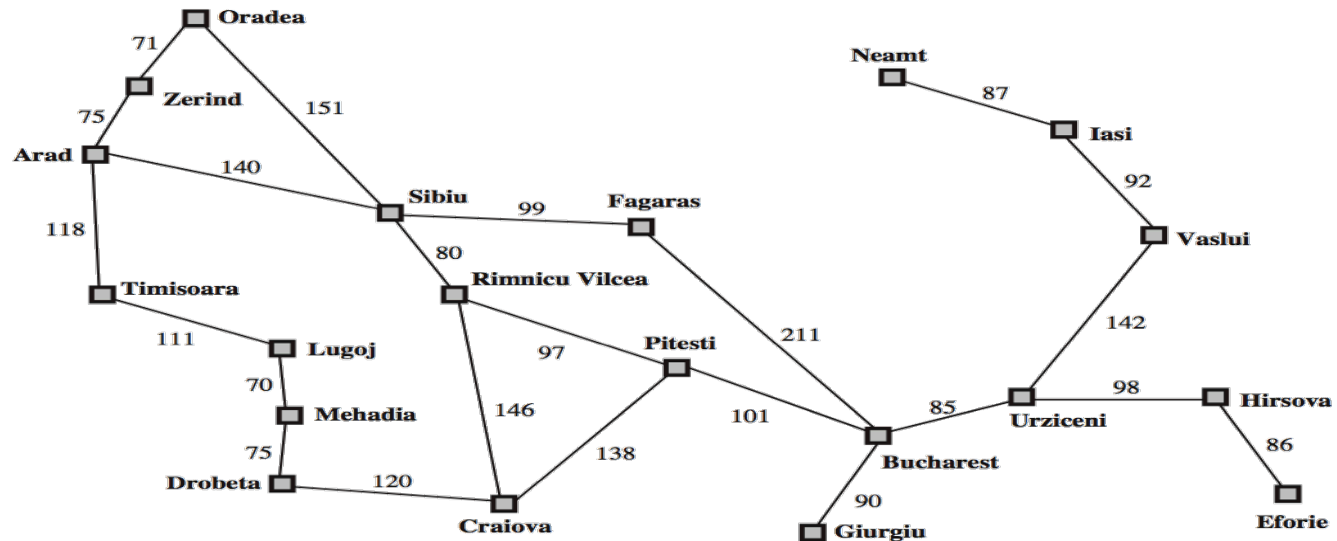
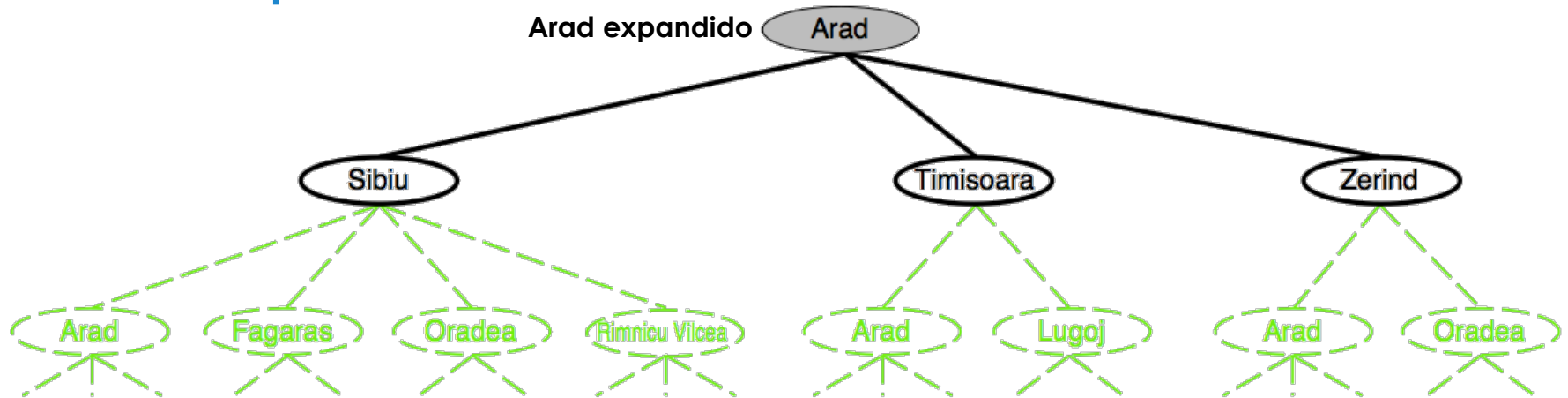
- Búsqueda como solución a problemas
 - Idea básica de la **Búsqueda en el espacio de estados**:
 - Simular la exploración del espacio de estados
 - Generando los sucesores de los estados ya explorados (Expandir estados).
 - La búsqueda genera un **árbol de búsqueda**
 - Nodo raíz = estado inicial.
 - **Nodos** (estados) y hojas (nodos sin hijos) generados por la función sucesor.
 - En general la búsqueda genera un **grafo de estados**. Se puede llegar a un estado por **múltiples caminos**.
 - Podemos volver al mismo estado después de visitarlo
 - Podemos encontrar diferentes caminos al mismo estado

Algoritmos básicos de búsqueda

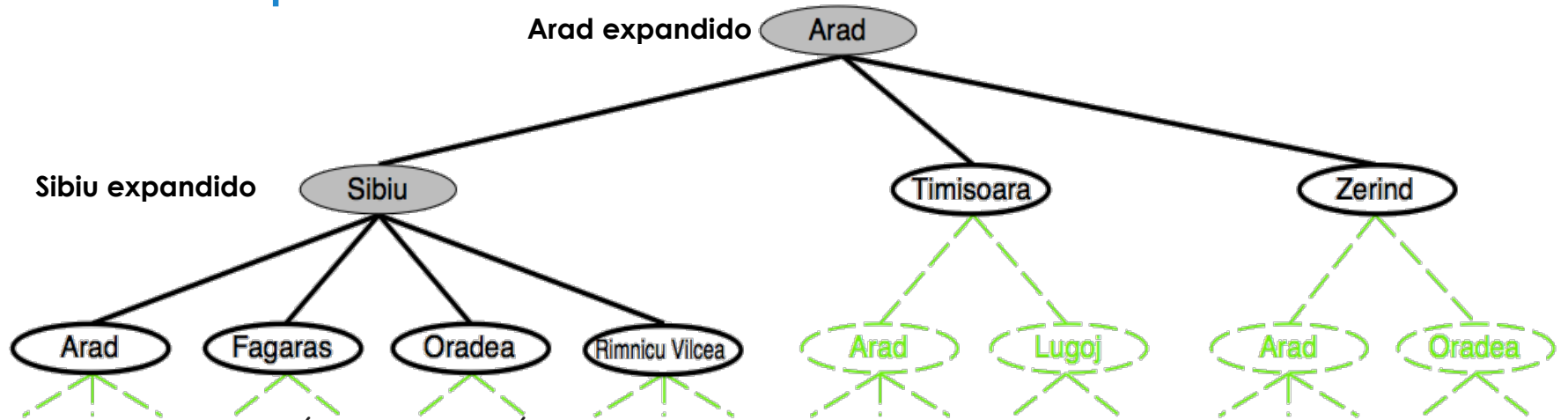
■ Nomenclatura

- La búsqueda genera un **árbol de búsqueda**
- **Nodo** representa estado
- **Expandir** estado es generar nuevos estados aplicando acciones permitidas sobre un estado.
- El nodo expandido es el **nodo padre** de los nodos resultantes (**nodos hijos**).
- Al conjunto de nodos sin hijos (no expandidos) se le denomina **frontera** o **lista abierta**.

Descripción informal de la búsqueda en árbol



Descripción informal de la búsqueda en árbol



function BÚSQUEDA-ÁRBOL (*problema*, *estrategia*) **returns** solución o fallo

Inicializa la *frontera* utilizando el estado inicial del problema

loop do

¡Determina el
proceso de
búsqueda!



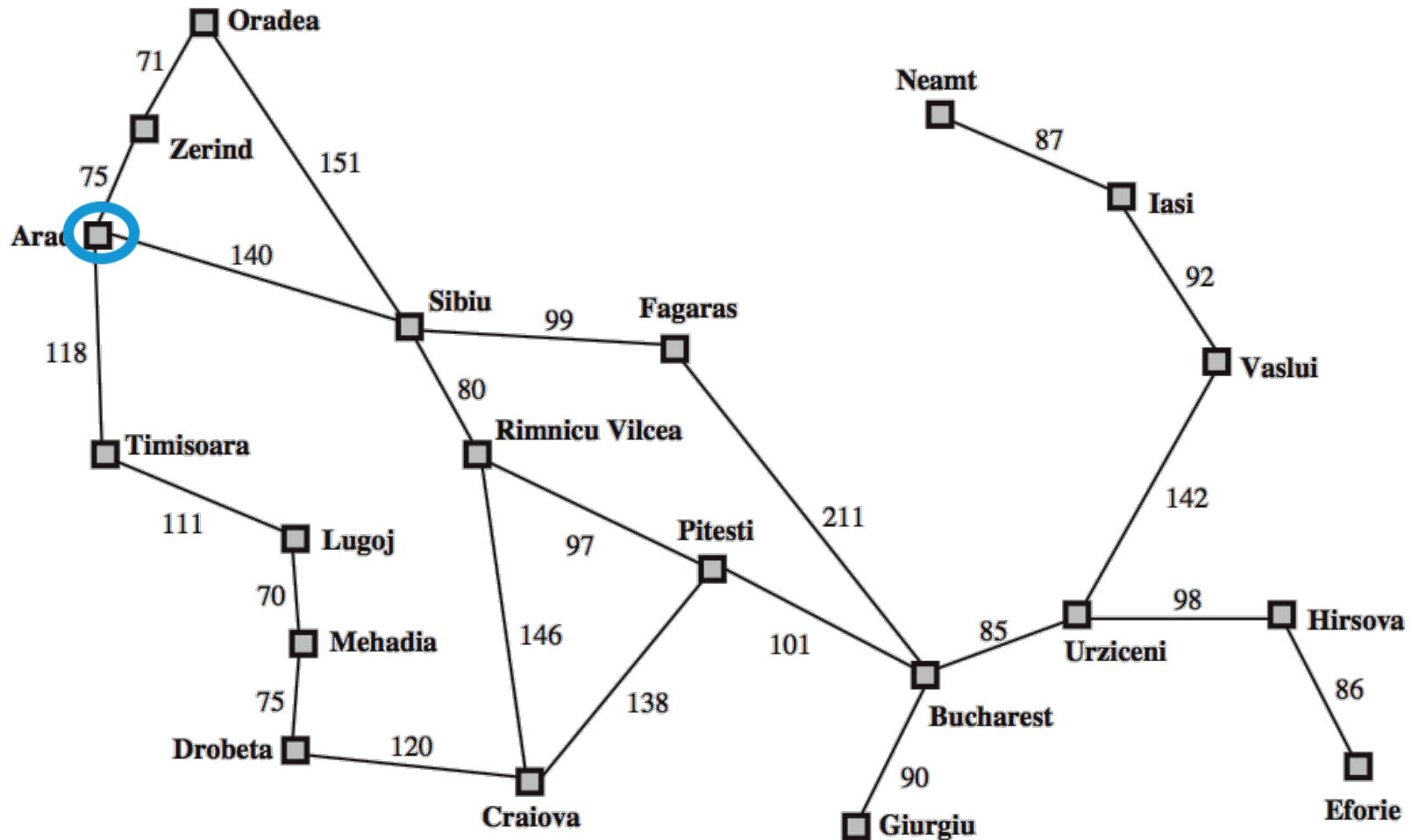
if la *frontera* está vacía **then return** fallo

elige un *nodo* hoja de la frontera de acuerdo a una *estrategia*

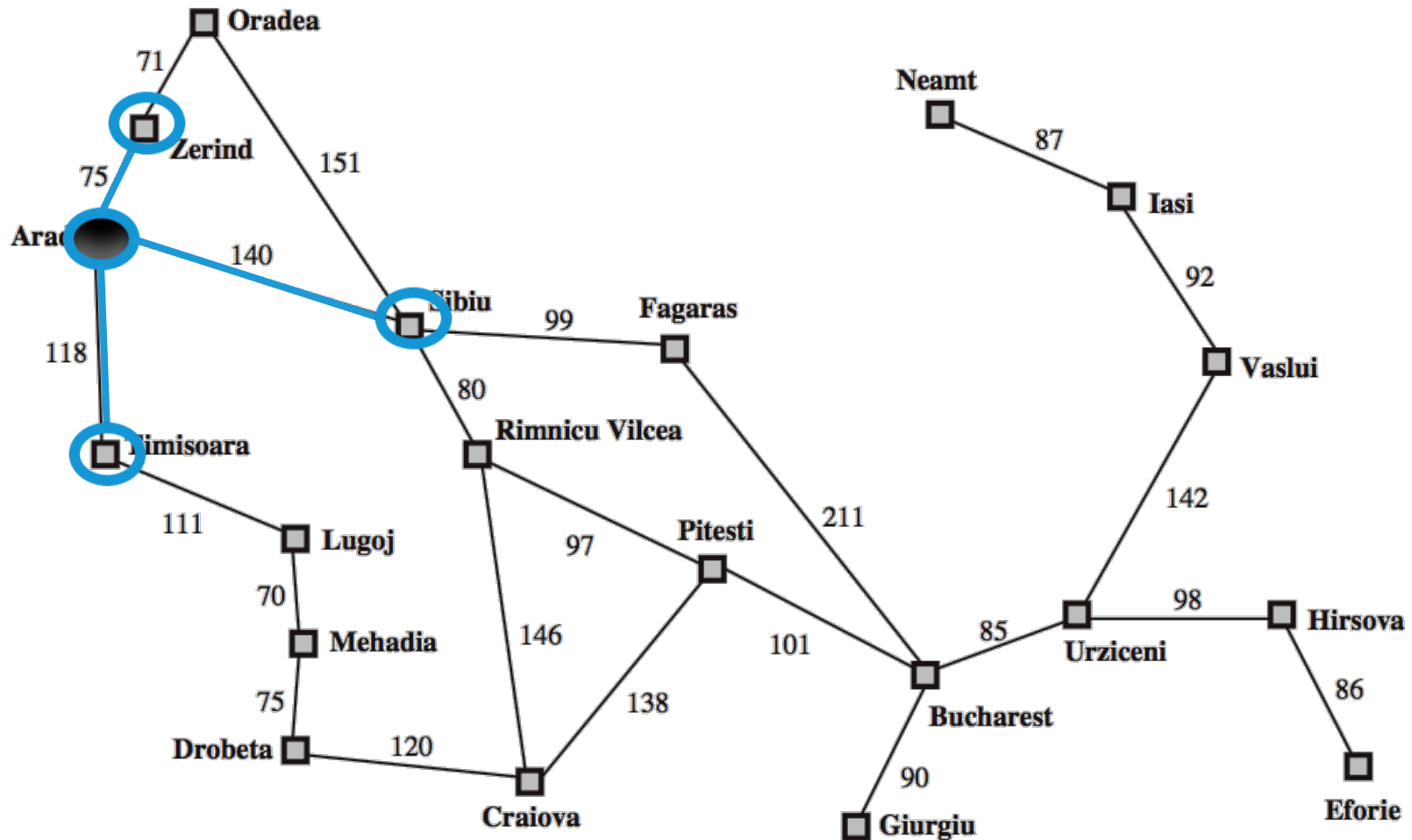
if el *nodo* contiene un nodo objetivo **then return** solución

expande el nodo elegido, añadiendo los nodos resultantes a la frontera

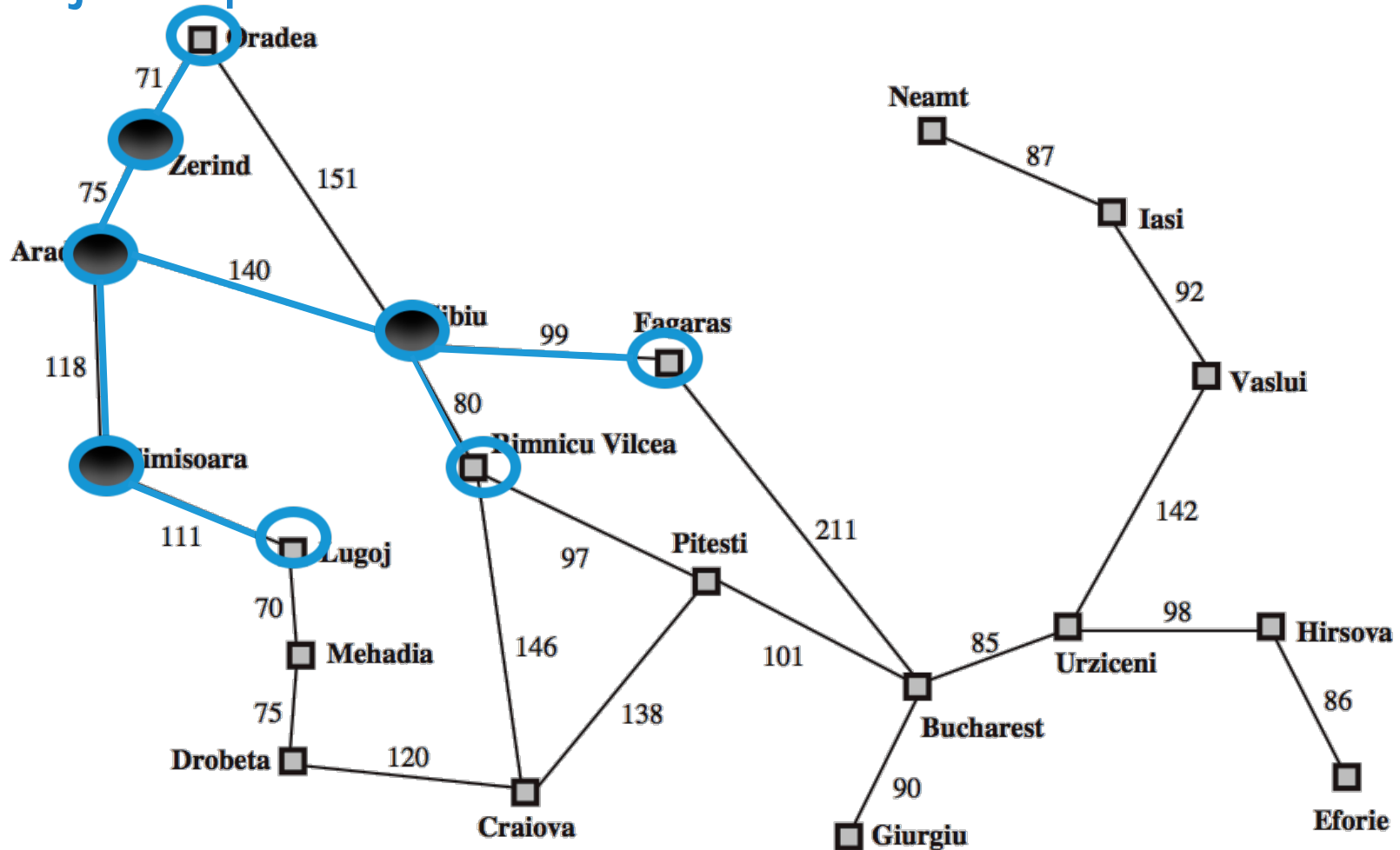
Ejemplo: Rumanía



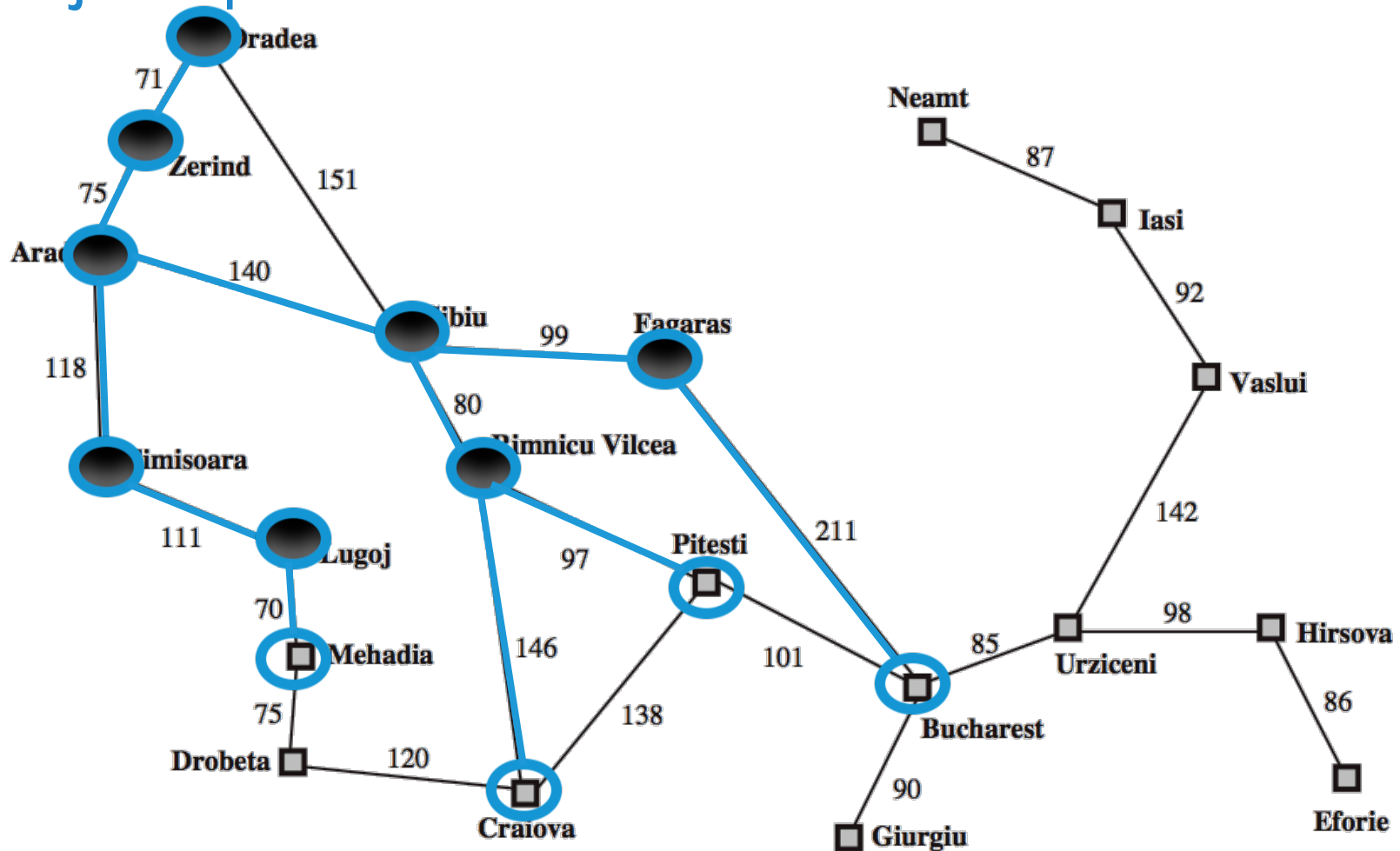
Ejemplo: Rumanía



Ejemplo: Rumanía

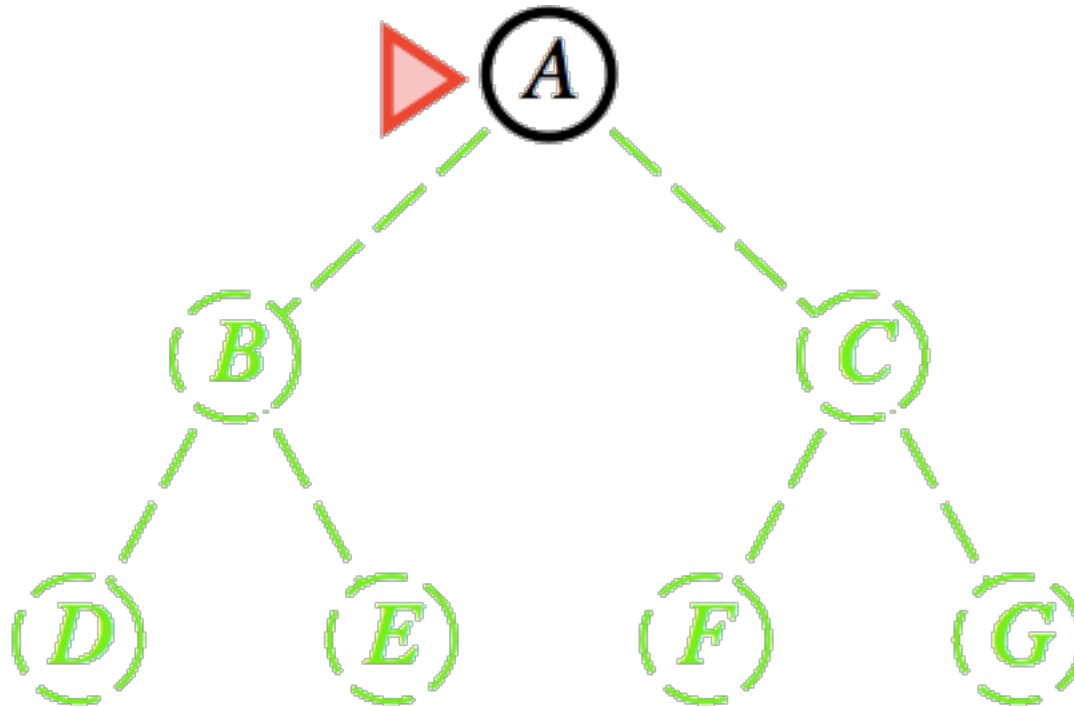


Ejemplo: Rumanía



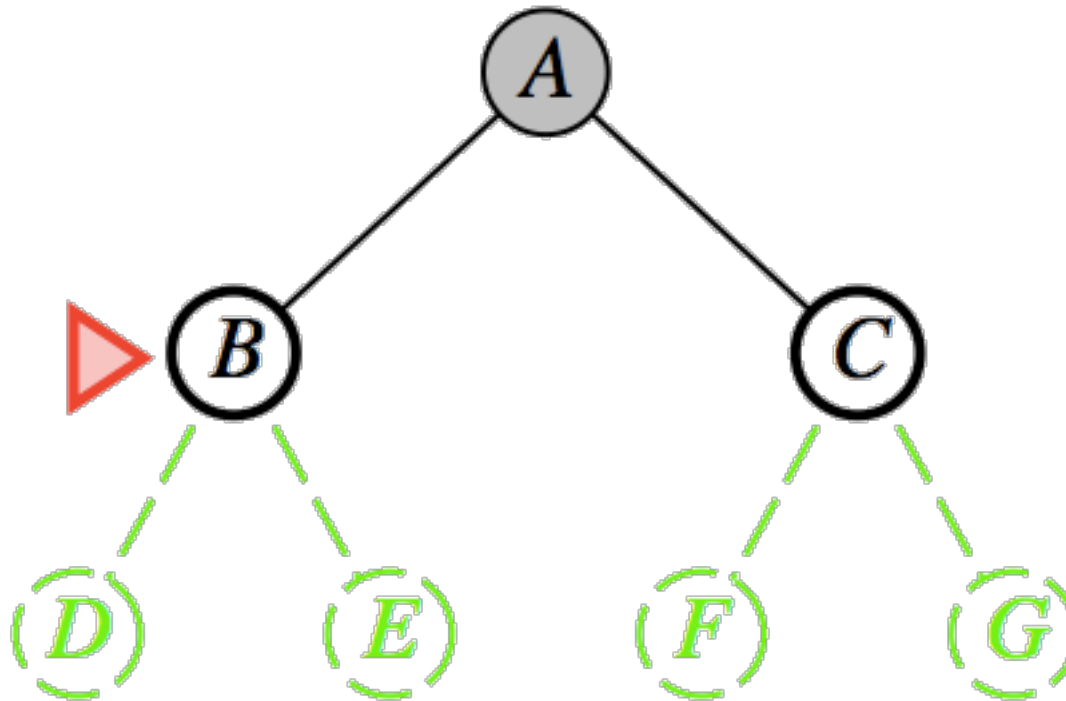
Primero en anchura, un ejemplo

- Expande el nodo no expandido menos profundo
- Implementación: *la frontera es una cola FIFO*



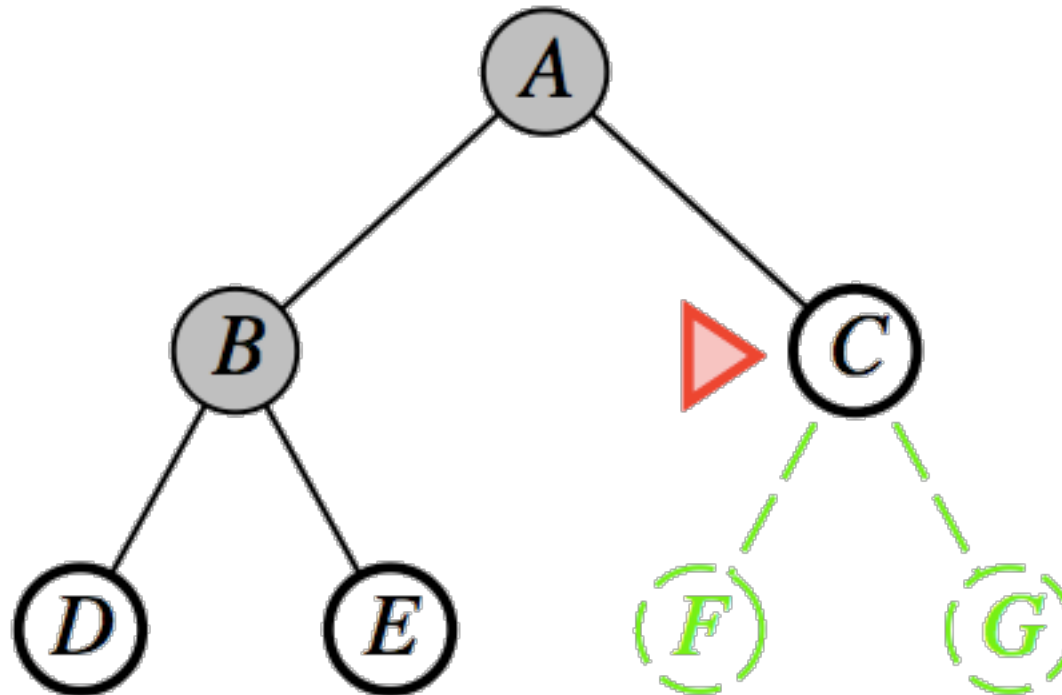
Primero en anchura, un ejemplo

- Expande el nodo no expandido menos profundo
- Implementación: *la frontera es una cola FIFO*



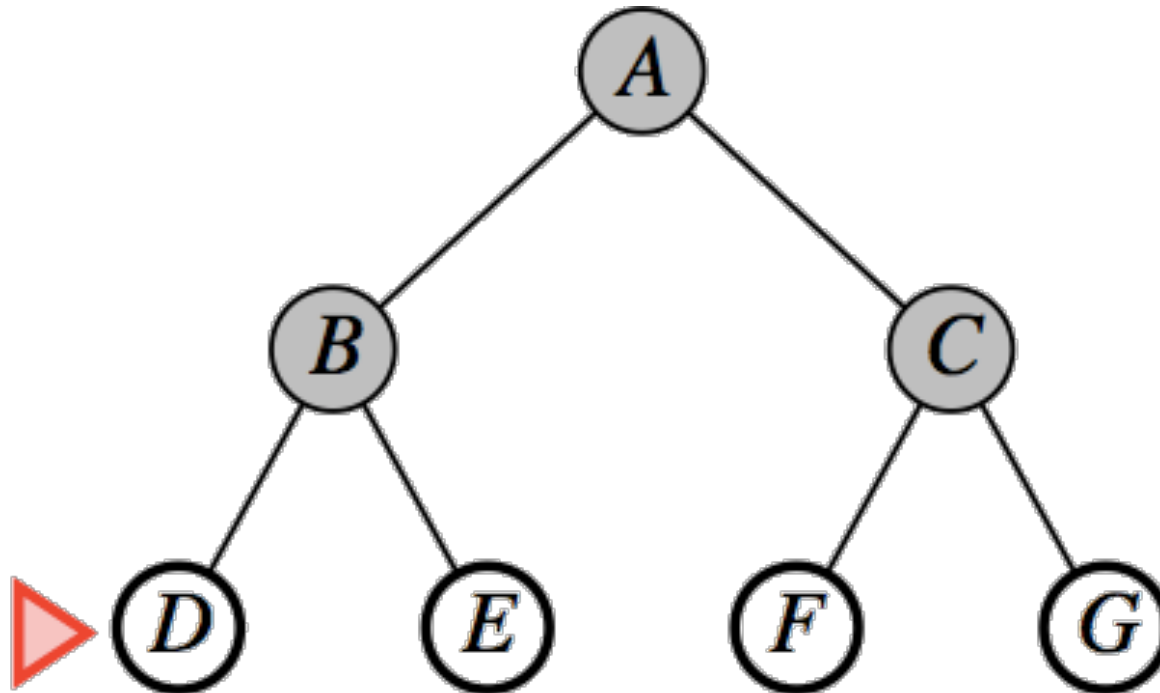
Primero en anchura, un ejemplo

- Expande el nodo no expandido menos profundo
- Implementación: *la frontera es una cola FIFO*



Primero en anchura, un ejemplo

- Expande el nodo no expandido menos profundo
- Implementación: *la frontera es una cola FIFO*



Primero en anchura, un ejemplo

Situación inicial, p.e.:

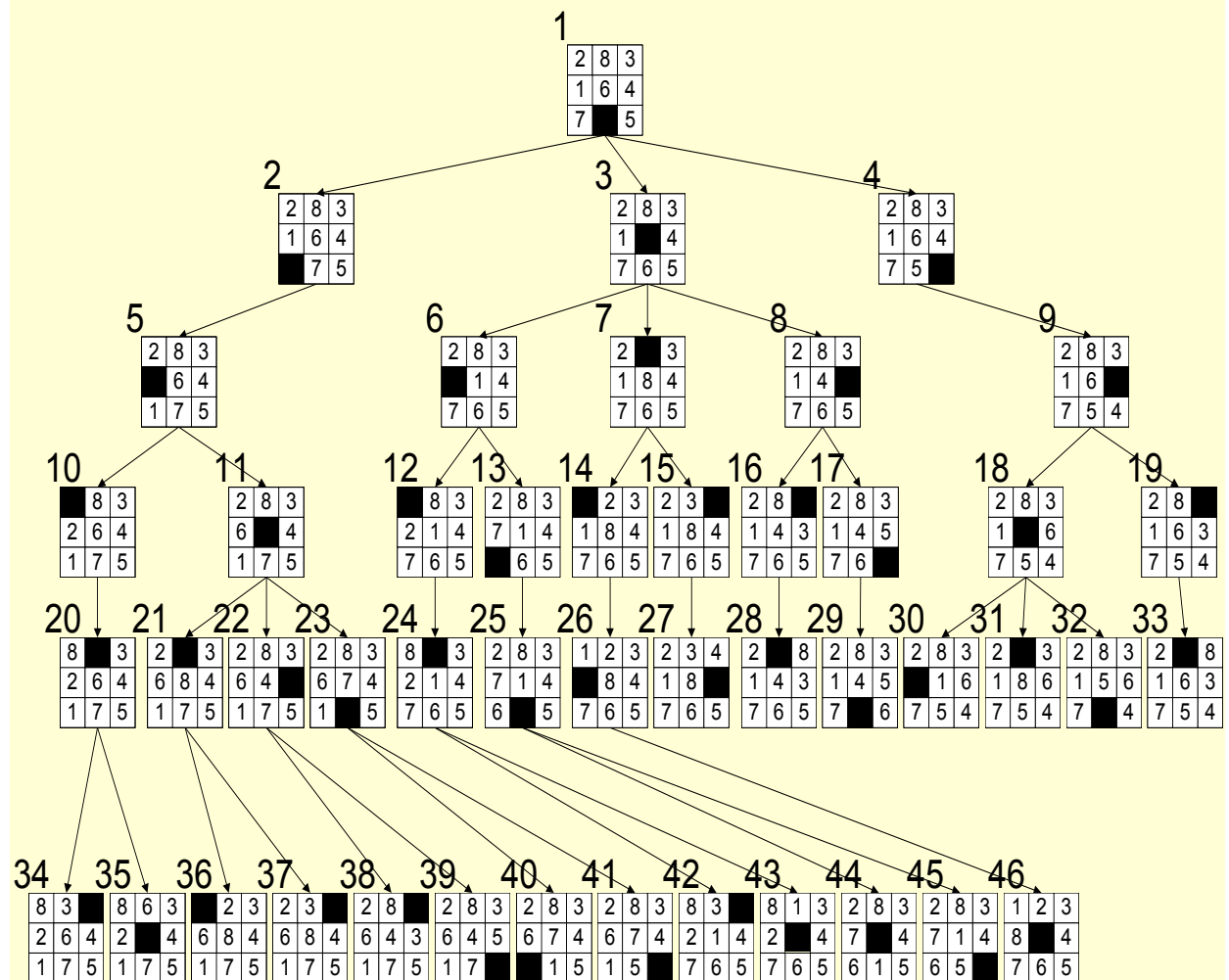
1	4	3
7		6
5	8	2

Estado objetivo

1	2	3
8		4
7	6	5

Reglas de juego

- R1: Mover hueco arriba
R2: Mover hueco abajo
R3: Mover hueco derecha
R4: Mover hueco izquierda



Búsqueda en anchura- evaluación

- Optimalidad:
 - *¿Encuentra siempre la solución optima?*
 - En general SI
 - Si el coste del camino es una función no decreciente de la profundidad.
 - El objetivo menos profundo no tiene porque ser el optimo si las acciones tienen diferente coste.

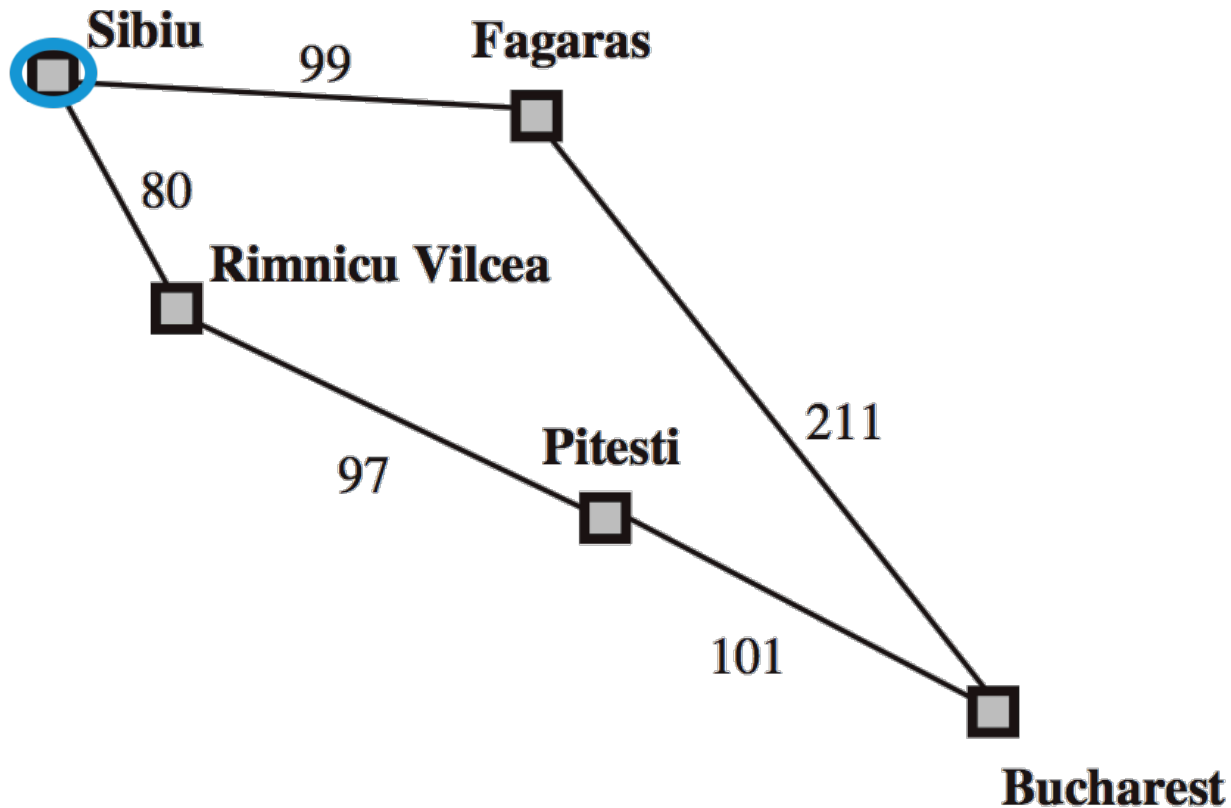
Búsqueda en anchura- evaluación

- Los **requisitos de memoria** son el problema
- La complejidad exponencial de los problemas de búsqueda no puede ser resuelta por la búsqueda ciega salvo para ejemplos pequeños.

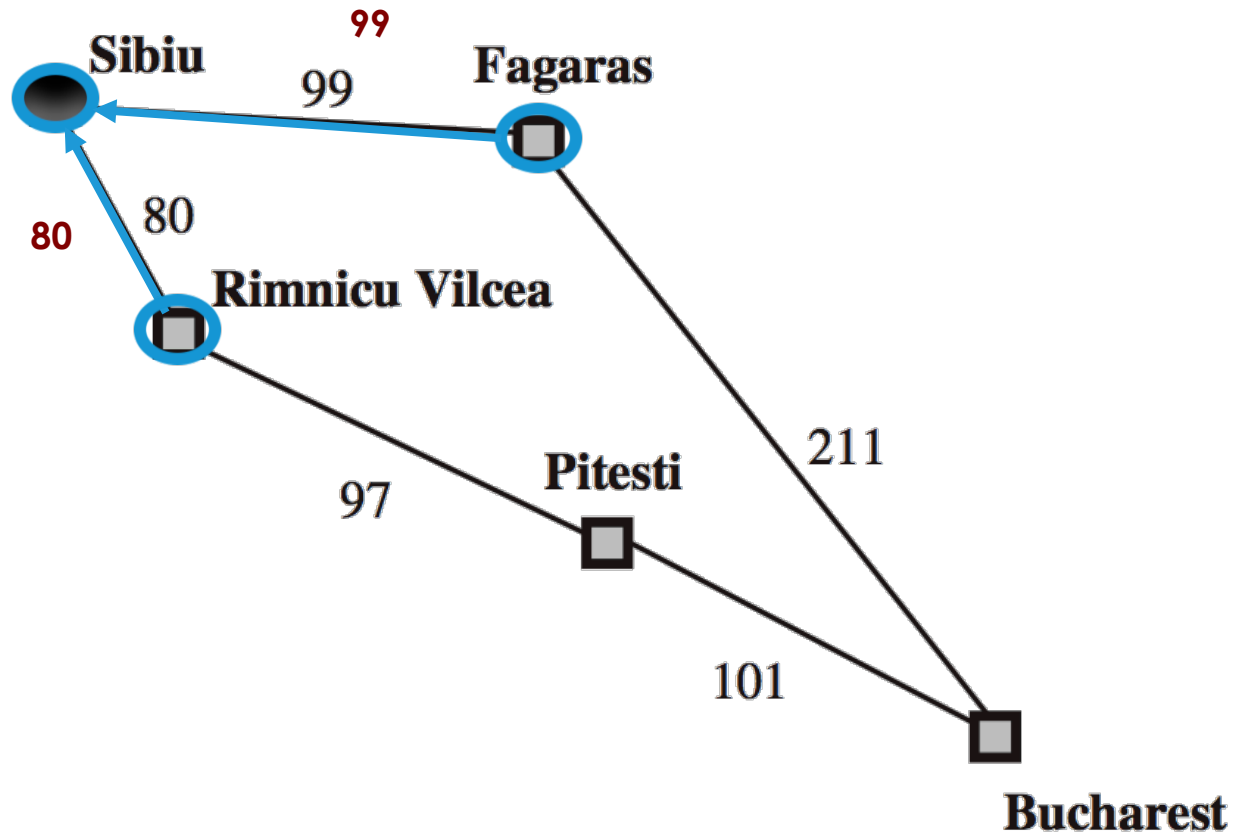


Profundidad	Nodos	Tiempo	Memoria
2	110	1.1 milisegundos	107 Kilobytes
4	11.110	111 milisegundos	10.6 megabytes
6	10^6	11 segundos	1 gigabyte
8	10^8	19 minutos	103 gigabytes
10	10^{10}	31 horas	10 terabytes
12	10^{12}	129 días	1 perabyte
14	10^{14}	35 años	99 petabytes
16	10^{16}	3500 años	1 exabyte
Complejidad temporal y espacial para una búsqueda en anchura de factor de ramificación $b=10$, 1 millón de nodos generados/segundo, 1000 bytes por nodo.			

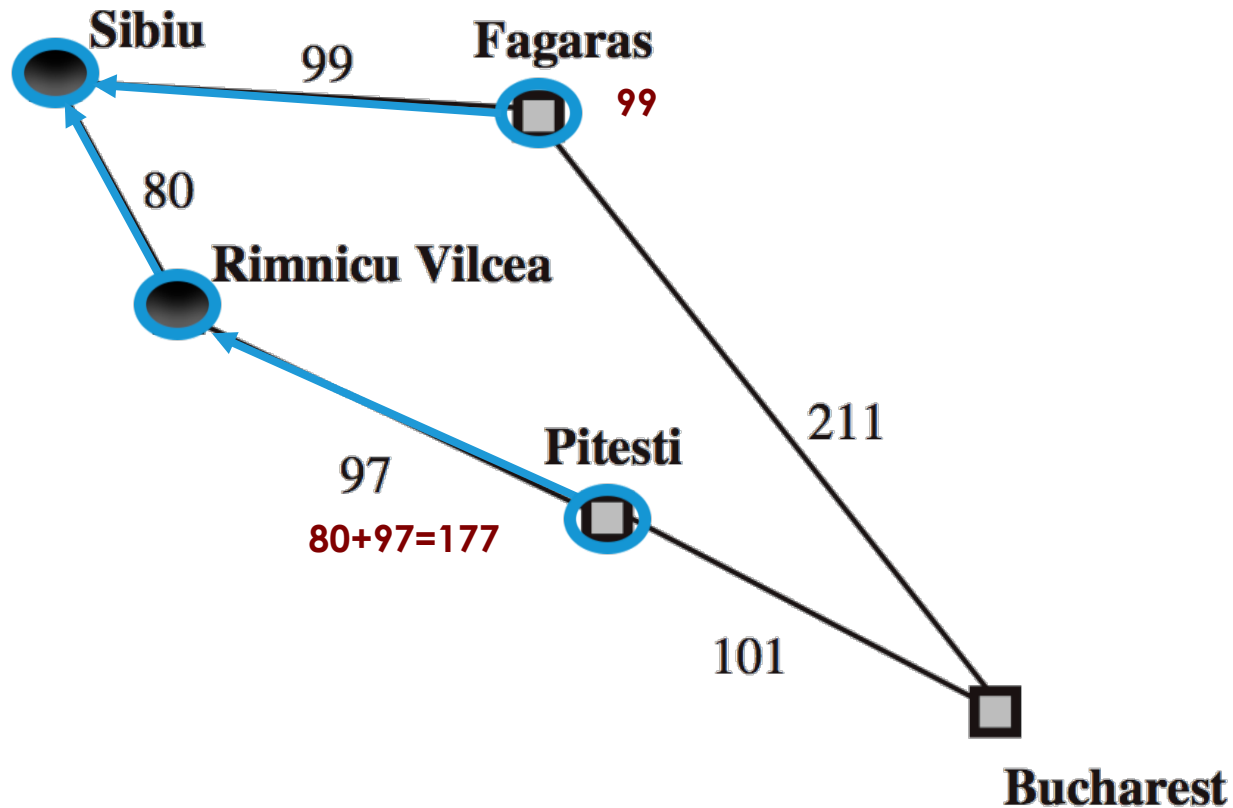
Búsqueda coste uniforme



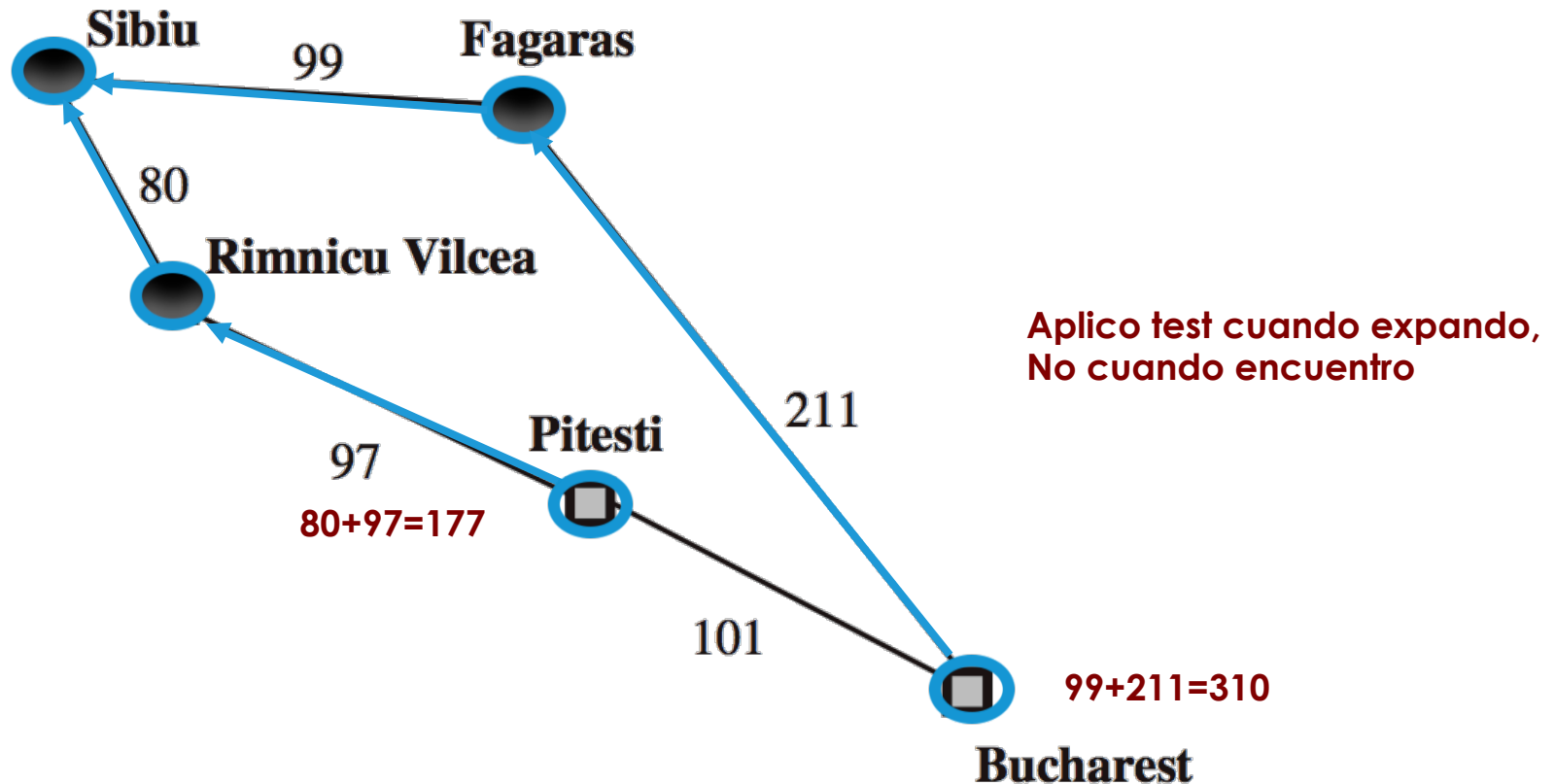
Búsqueda coste uniforme



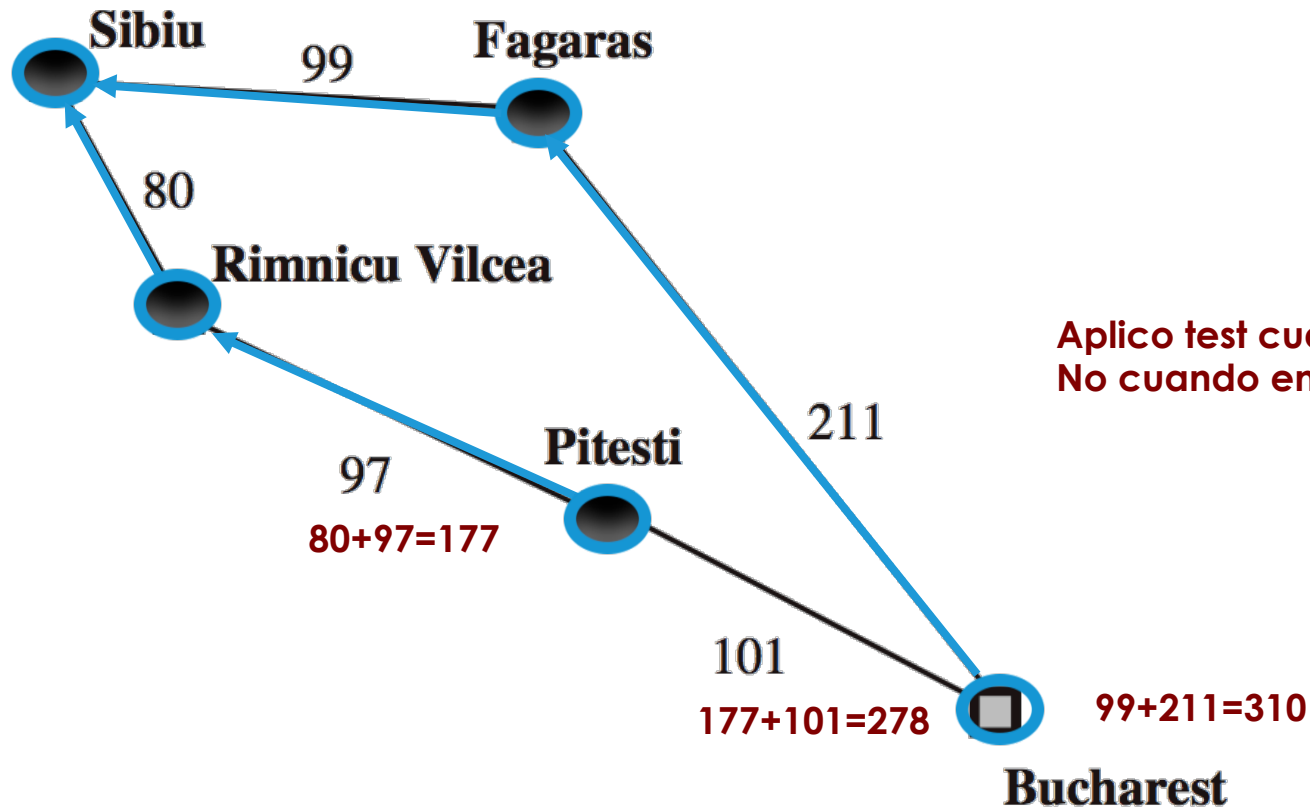
Búsqueda coste uniforme



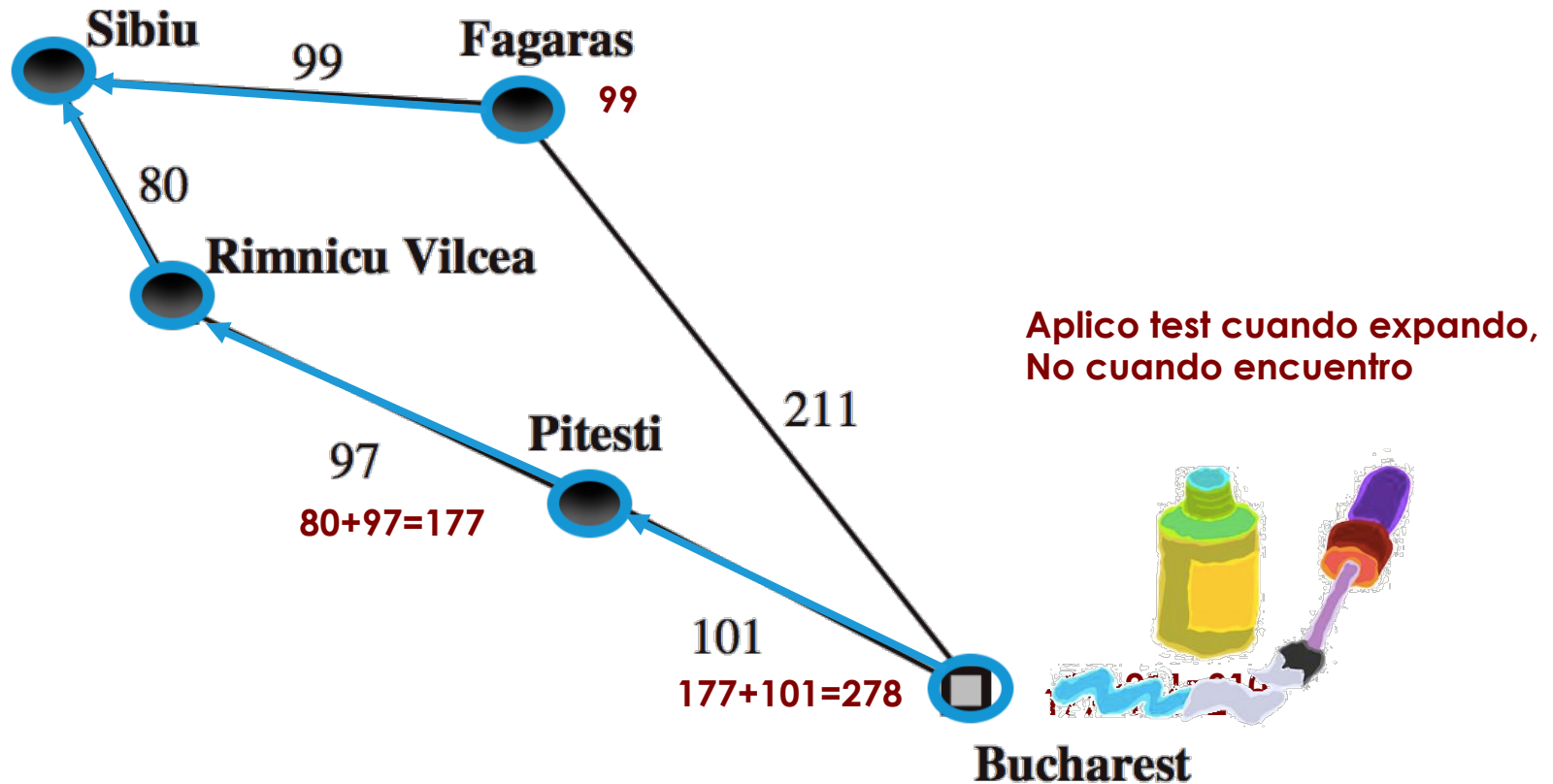
Búsqueda coste uniforme



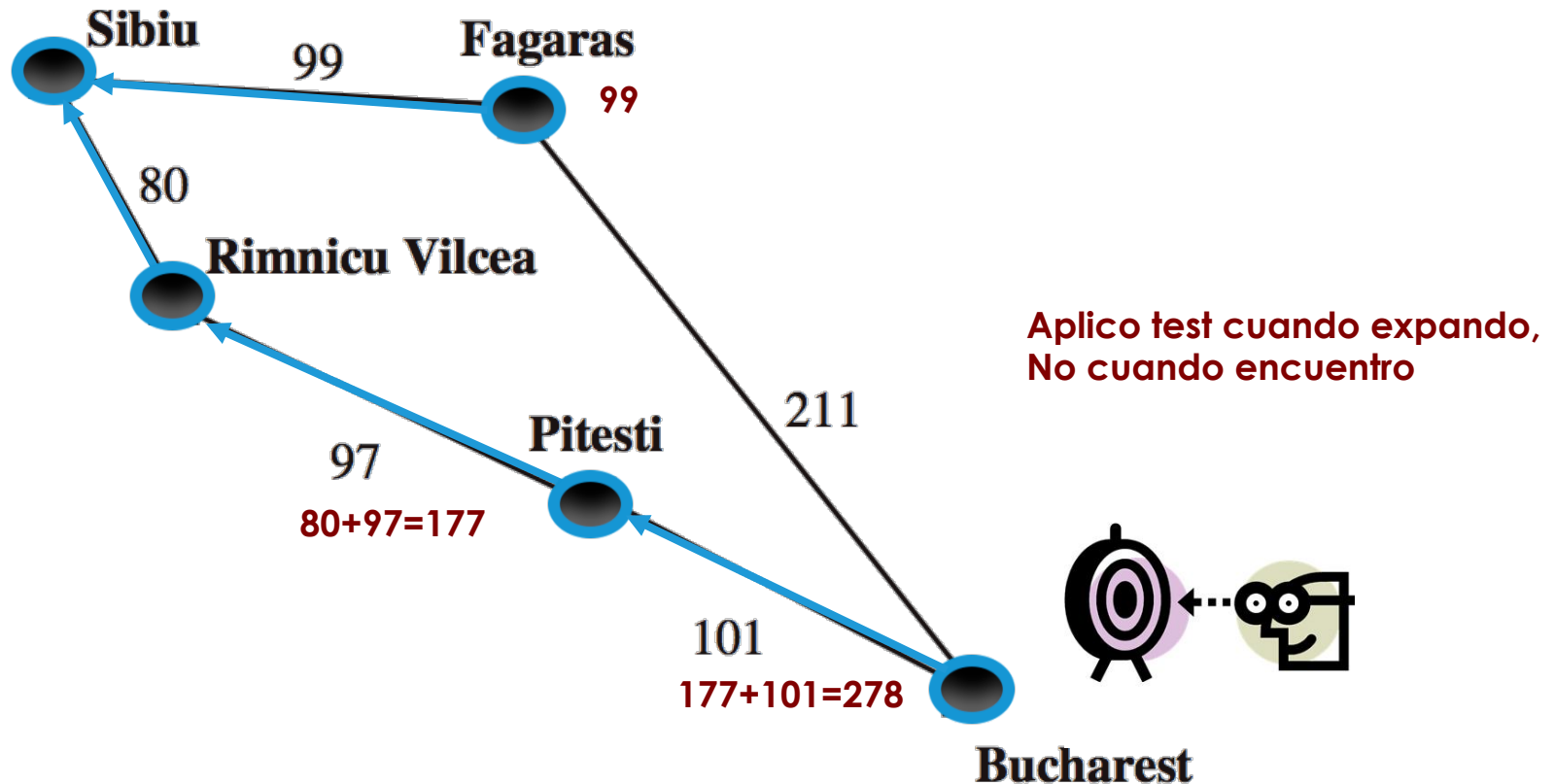
Búsqueda coste uniforme



Búsqueda coste uniforme



Búsqueda coste uniforme



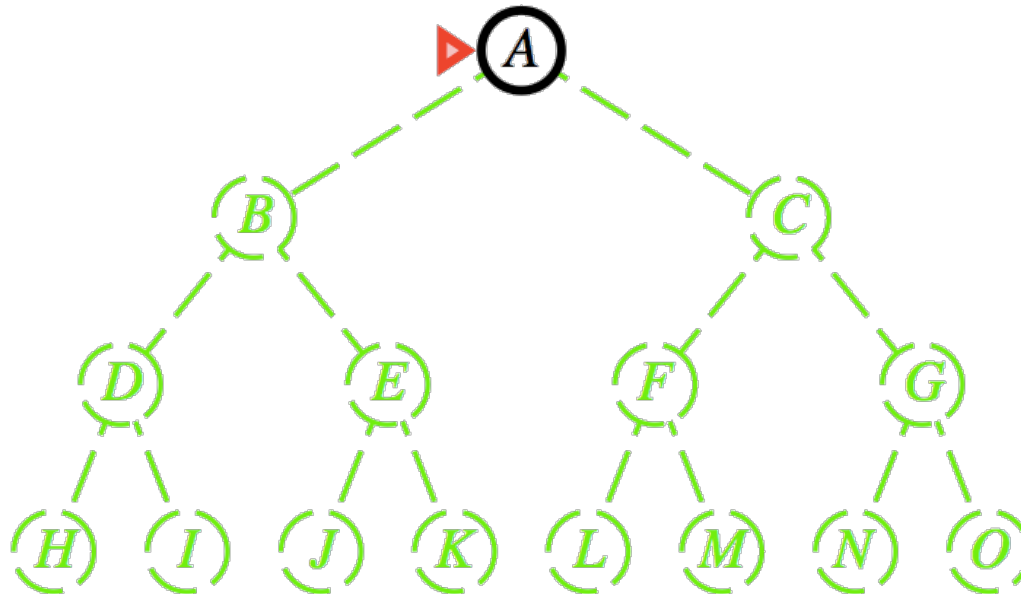
Evaluación: Coste uniforme

Es óptima

- Cuando elegimos un nodo para expandir, el camino óptimo a ese nodo ya ha sido encontrado
 - Si el nodo a expandir n no tuviera el menor coste, tendría que haber otro nodo en la frontera n' **en el camino óptimo del inicio a n** , pero entonces tendría menor coste y debería haber sido seleccionado antes.
- Como los pasos son no negativos, los caminos nunca tienen menor coste al añadir un nodo.
- La búsqueda uniforme expande nodos en orden de sus caminos óptimos

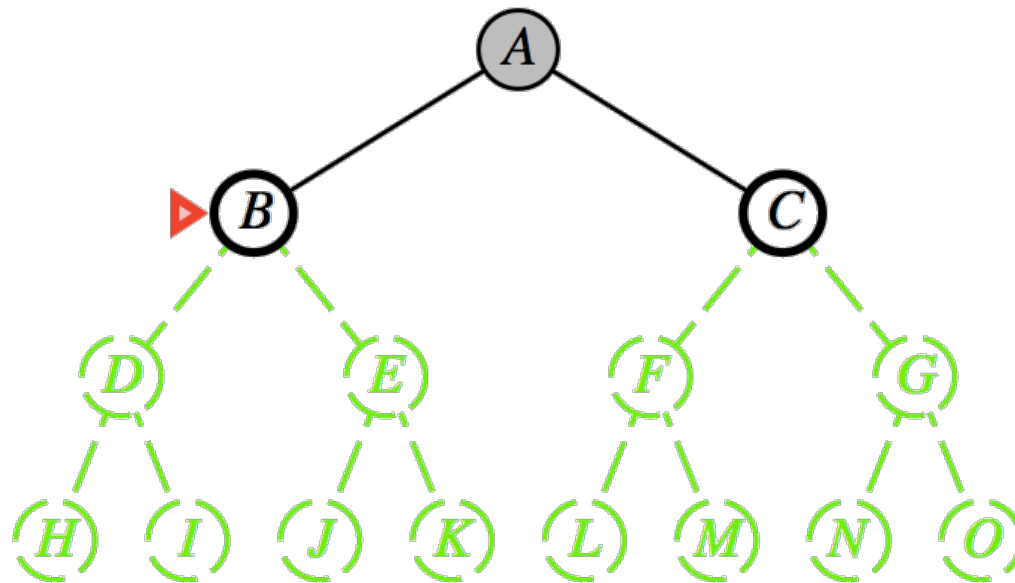
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



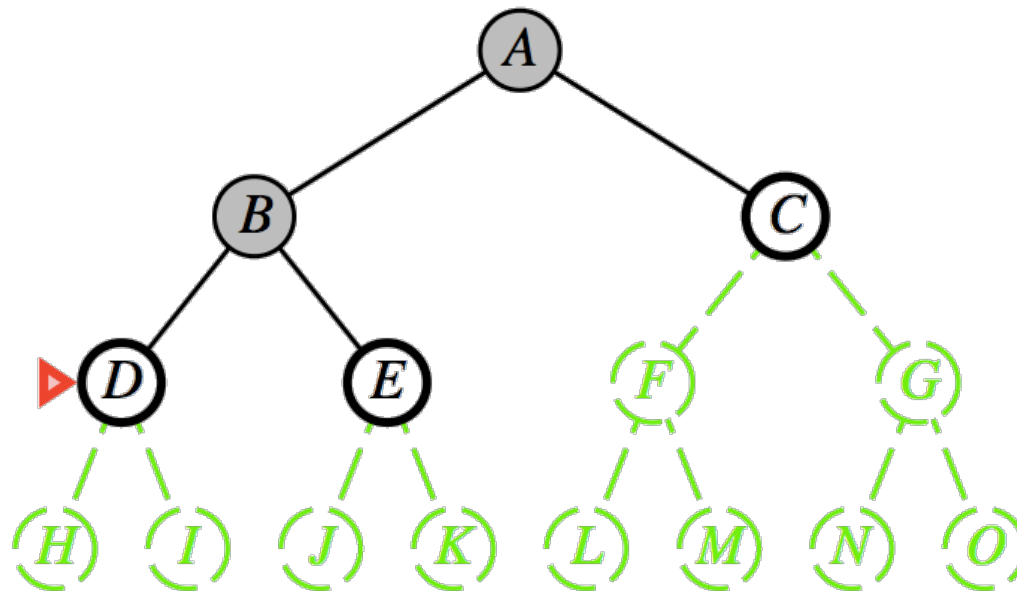
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



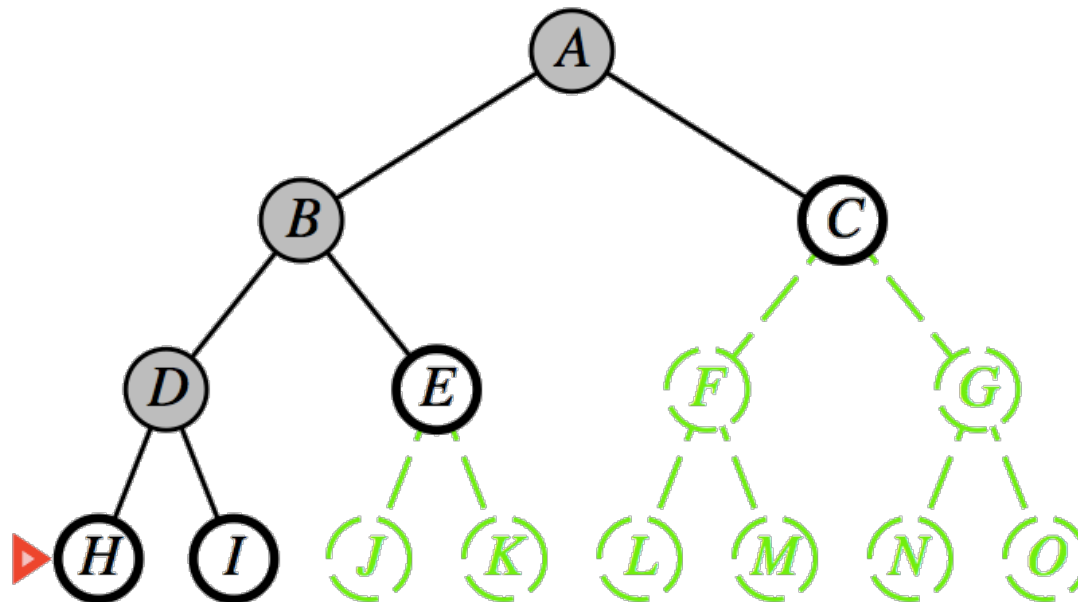
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



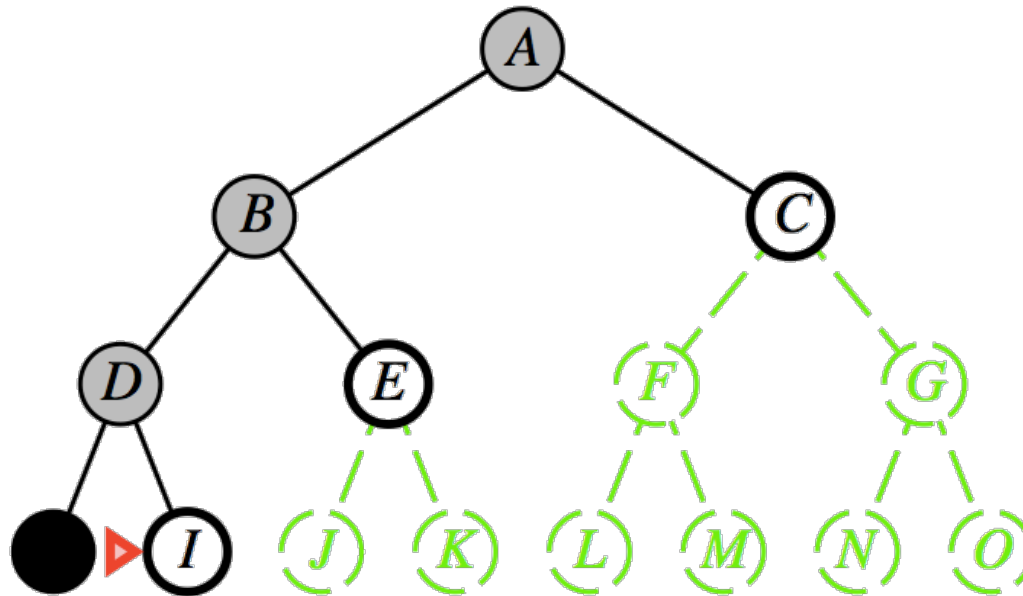
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



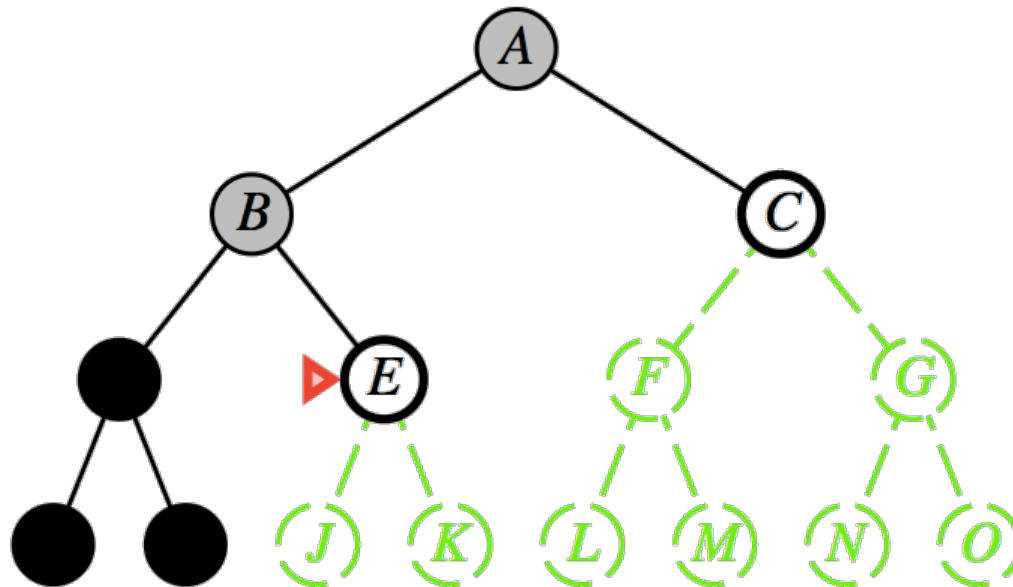
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



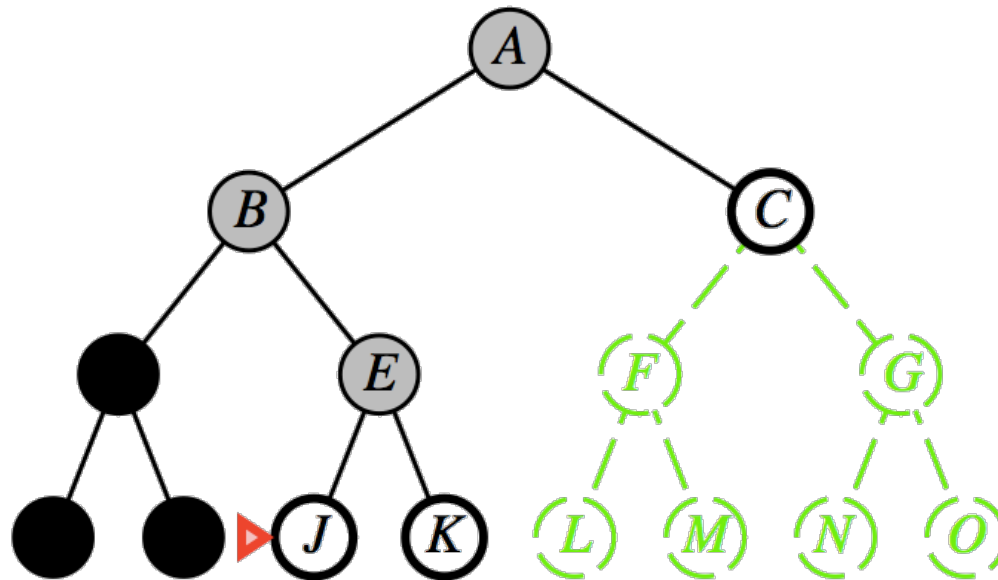
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



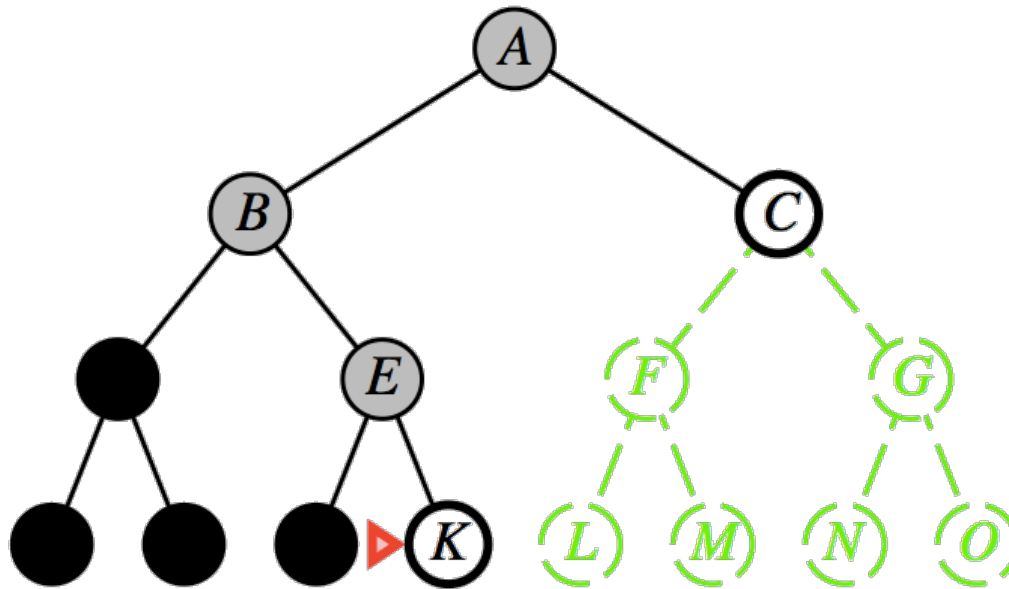
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



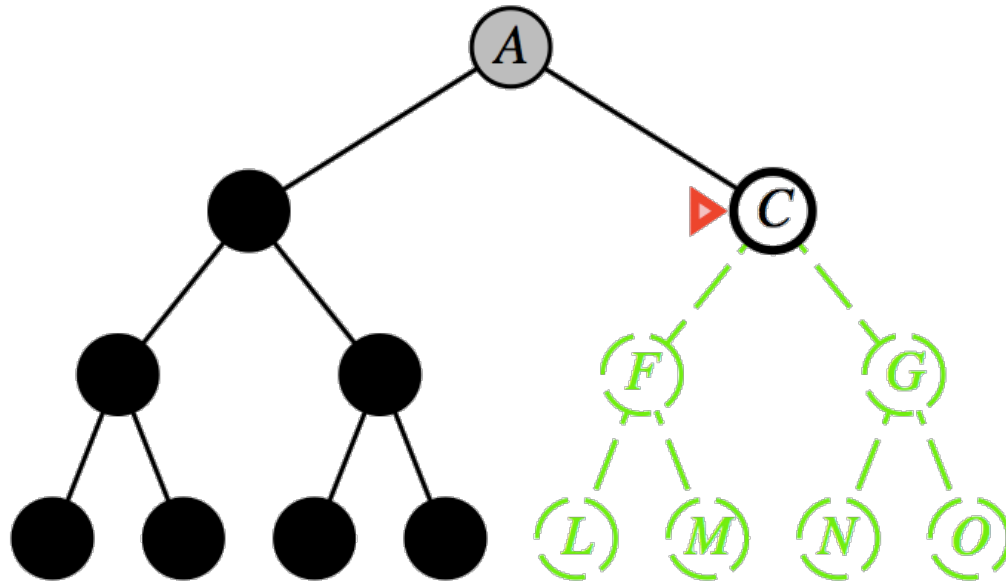
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



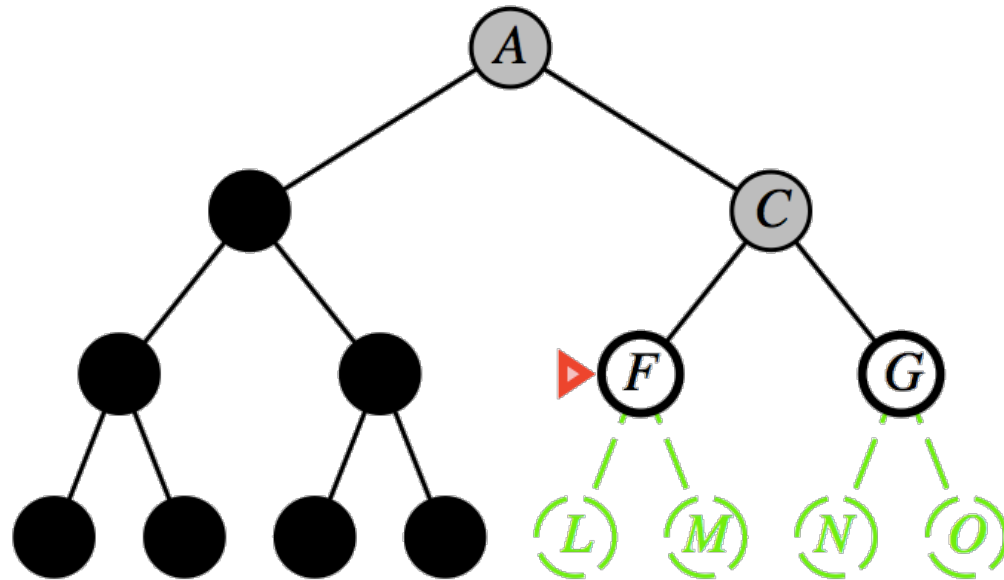
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



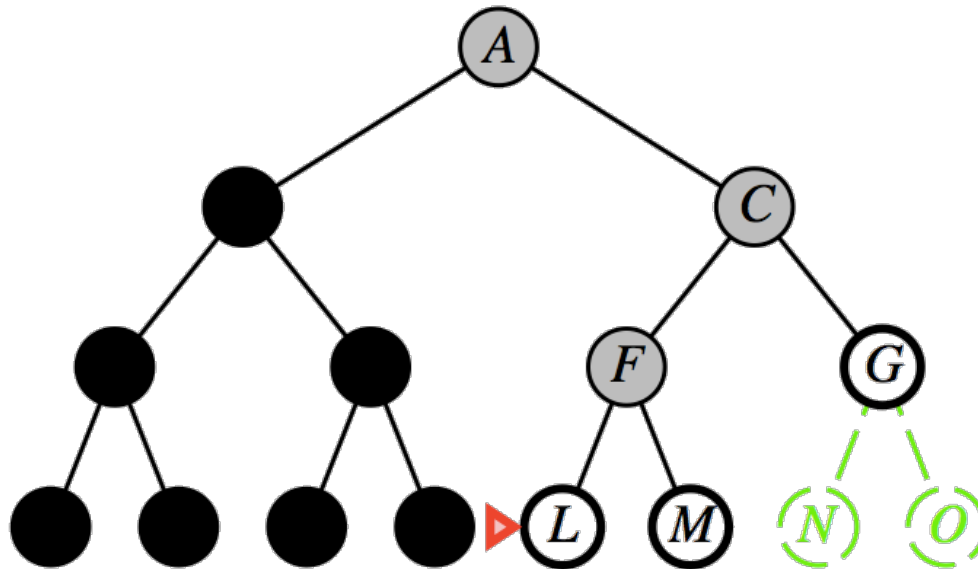
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



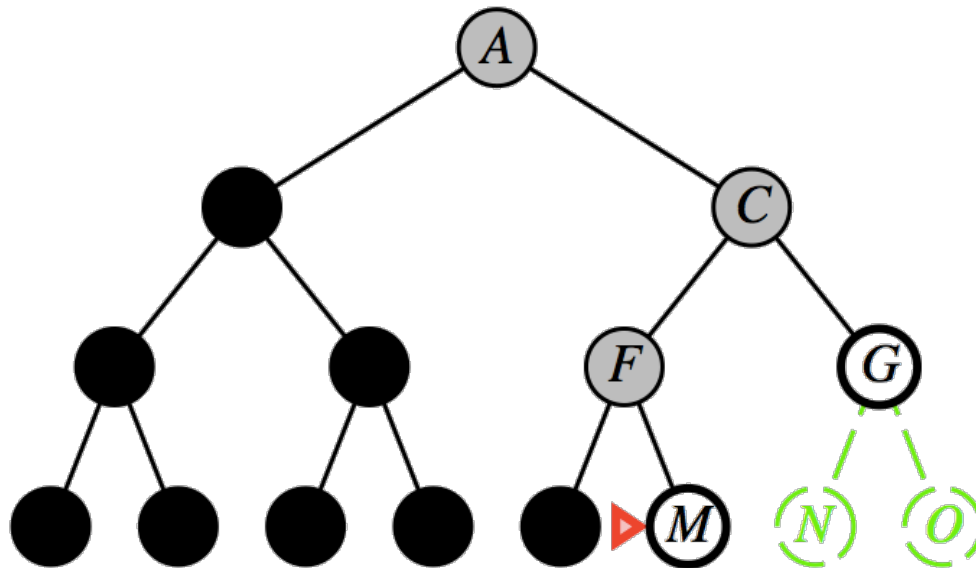
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



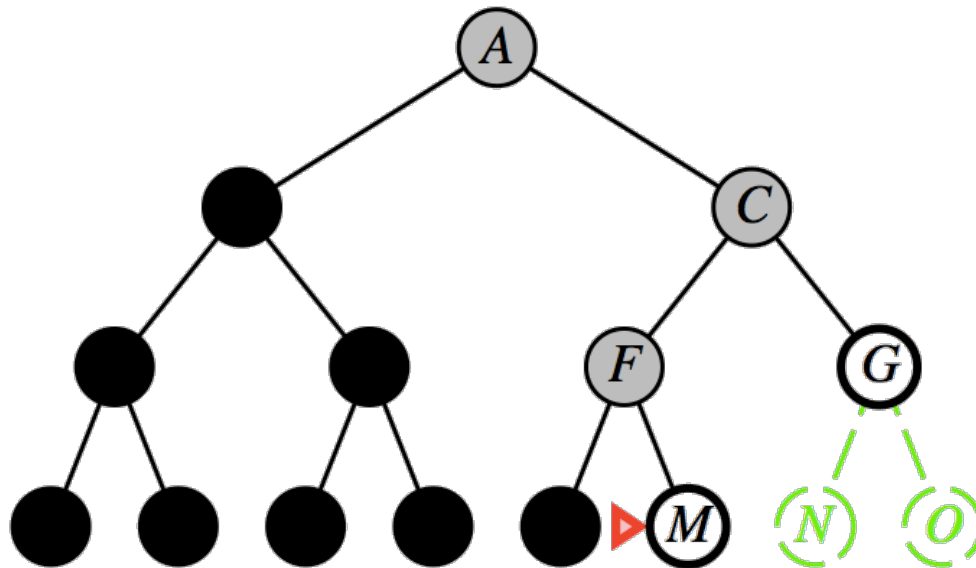
Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



Primero en profundidad, Ejemplo

- Expande nodo más profundo primero
- Implementation: *frontera* es una LIFO



Búsqueda profundidad iterativa (Iterative Deepening, ID)



- Intenta **combinar** el comportamiento **espacial de la búsqueda en profundidad** con la **optimalidad de la búsqueda en anchura**
- El algoritmo consiste en **realizar búsquedas en profundidad sucesivas** con un nivel de profundidad máximo acotado y creciente en cada iteración
- Así se consigue el **comportamiento de la búsqueda en anchura** pero **sin su coste espacial**, ya que la exploración es en profundidad, y además los nodos se regeneran a cada iteración
- Además esto permite evitar los casos en que la búsqueda en profundidad no acaba (existen ramas infinitas)
- En la primera iteración la profundidad máxima será 1 y este valor irá aumentando en sucesivas iteraciones hasta llegar a la solución
- Para garantizar que el algoritmo acaba si no hay solución, se puede definir una cota máxima de profundidad en la exploración

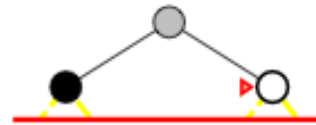
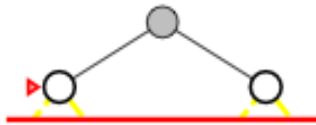
ID-search, ejemplo

■ Límite=0



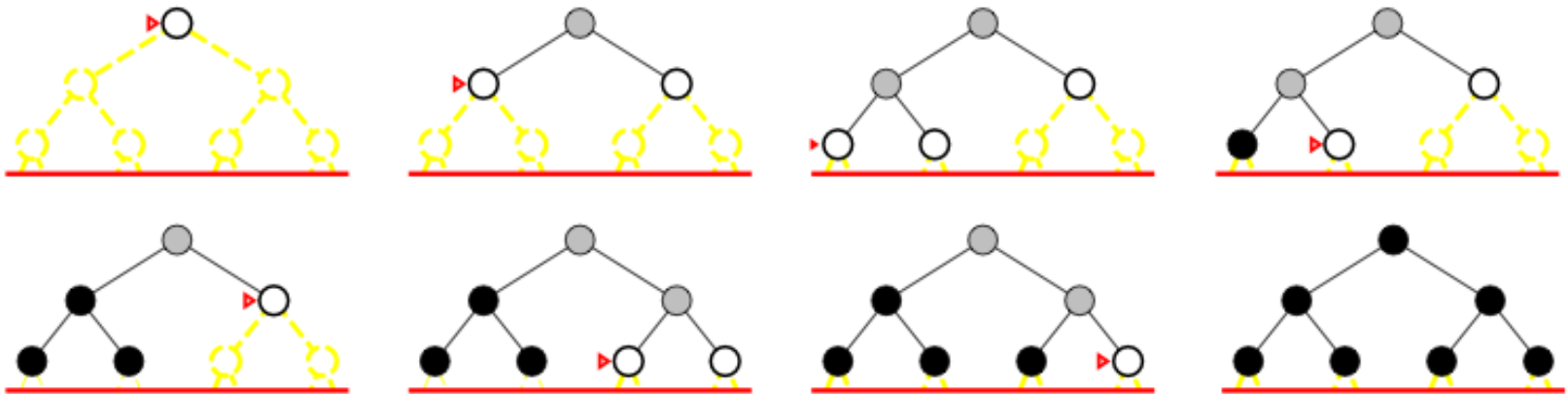
ID-search, ejemplo

■ Limite=1



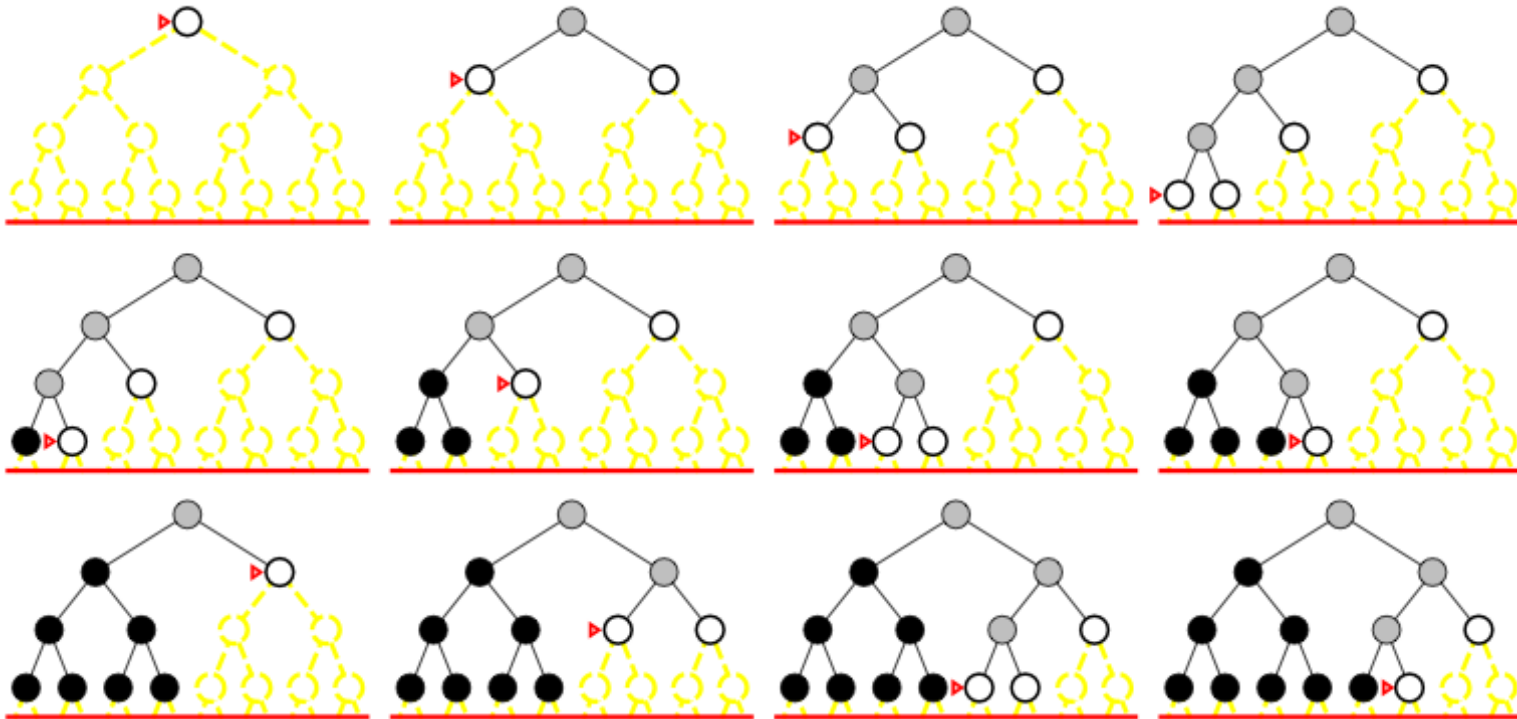
ID-search, ejemplo

- Límite=2

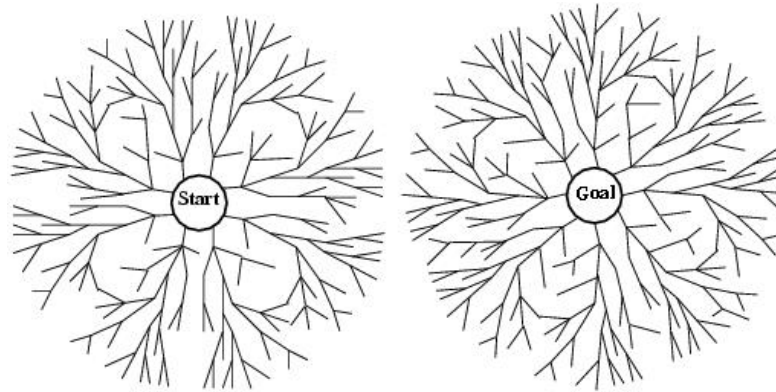


ID-search, ejemplo

■ Límite=3



Búsqueda bidireccional



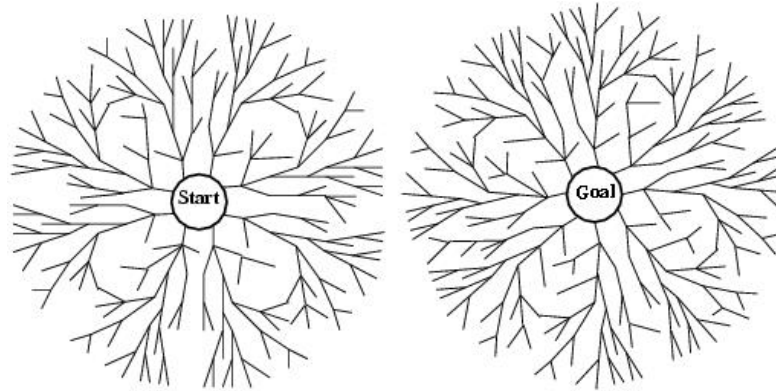
- Búsquedas simultáneas desde el objetivo y el inicio.

- Motivación:

$$b^{d/2} + b^{d/2} \neq b^d$$

- Comprueba si el nodo pertenece a la otra frontera antes de expandir.
- La complejidad espacial es el punto débil
- Completo y óptimo si ambas búsquedas son en anchura.

¿Cómo buscar hacia atrás?



- Tel predecesor de cada nodo debe ser computable fácilmente.
 - Cuando las acciones son reversibles

Resumen de algoritmos

Criterio	Breadth-First	Uniform-cost	Depth-First	Depth-limited	Iterative deepening	Bidirectional search
Compleitud	SI ^a	SI ^{a,b}	NO	SI, si $l \geq d$	SI ^a	SI ^{a,d}
Temporal	$O(b^{d+1})$	$O(b^{l+[C^*/e]})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Espacial	$O(b^{d+1})$	$O(b^{l+[C^*/e]})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimalidad	SI ^c	SI	NO	NO	SI ^c	SI ^{c,d}

b factor de ramificación, **d** profundidad de la solución de menor profundidad, y **m** máxima profundidad de l árbol de búsquedas, **l** límite de la profundidad.

^a completa si b es finito

^b completo si el coste de un paso es $\text{coste} > e$ para e positivo

^c optimo si todos los costes en cada paso son iguales

^d si ambas direcciones usan búsqueda en anchura



Doctorado Ciencias Cognitivas

(

